NutriVision

AI-Powered Indian Food Nutrition Estimator

Final Technical Report · Milestones 6

Group	Group 2 — Data Science & AI Lab
Institute	IIT Madras
Project	NutriVision – AI Food Analyzer
Members	Sahil Raj · Sahil Sharma · Aman Mani Tiwari · Samreen Fathima · Tasneem Shahnawaz
Live Demo	https://7860-01kp8gvr84ng3myeemyw7vzz46.cloudspaces.litng.ai (Primary — more credits)
Mirror URL	https://7860-01knjbx5j5gnvmttvxykcmx4bh.cloudspaces.litng.ai/
Stack	Python · PyTorch · timm · HuggingFace SAM3 · Gradio · LangChain · Ollama LLM

1. Introduction & Problem Statement

NutriVision is an end-to-end AI system that accepts a photograph of a meal — including complex Indian multi-dish plates — and returns a detailed nutritional breakdown with personalised dietary suggestions. The system is specifically designed to address the limitations of existing nutrition tracking tools, which require manual food entry, assume fixed 100 g portions, and have poor coverage of Indian cuisines.

The core technical challenge is two-fold: (1) accurately identifying multiple food items in a single photograph, and (2) estimating the real-world portion weight of each identified item from 2-D pixel data alone, without depth sensors or specialised hardware. Downstream of these two problems, nutritional values are retrieved from a structured database and personalised recommendations are generated using a large-language-model pipeline grounded in ICMR dietary guidelines.

1.1 Objectives

- Develop a multi-food detection/classification system for Indian cuisine.
- Estimate portion size from a single photograph using geometry and density priors.
- Map detected foods to a structured nutritional database and compute macro/micronutrient intake.
- Perform personalised nutritional gap analysis against ICMR Recommended Dietary Allowances.
- Generate dietary suggestions via a RAG-augmented LLM pipeline (Gemini + MedGemma).
- Integrate all components into a deployable end-to-end application.

1.2 Comparison with Existing Solutions

The table below summarises how NutriVision addresses gaps in existing tools.

Feature	Traditional Apps	Basic Classifiers	NutriVision
Manual food entry required	Yes	No	No
Multi-food detection in one image	No	Limited	Yes (SAM3 + Classifier)
Portion size estimation from image	No	No (100 g fixed)	Yes (PCA + density)
Indian multi-dish meal support	Limited	Limited	Full support

Personalised nutrient gap analysis	Limited	No	Yes (ICMR RDA)
Conversational AI dietary suggestions	No	No	Yes (Gemini + MedGemma)

1.3 Key Contributions

- Multi-food segmentation using SAM3 — handles overlapping Indian thali-style plates.
- Fine-grained 80-class Indian food classifier (ConvNeXtV2-Tiny, 97.92% Top-1 accuracy).
- Coin-calibrated PCA geometry for rotation-invariant real-world dimension estimation.
- Density-enforced LLM weight estimation — LLM handles geometry, JSON enforces density.
- Nutrition scaling against ICMR RDA + RAG-grounded Gemini/Ollama suggestions.
- Production Gradio web app deployed on Lightning AI with streaming step progress.

2. Deployed Web Application

🌐 **Primary URL:** https://7860-01kp8gvr84ng3myeemyw7vzz46.cloudspaces.litng.ai

🔶 **Mirror URL:** https://7860-01knjbx5j5gnvmttvxykcmx4bh.cloudspaces.litng.ai/

Both URLs serve the identical Gradio application. The primary URL has more Lightning AI compute credits for uninterrupted GPU inference. The application runs on Lightning AI's cloud platform with GPU acceleration and is built with Gradio's streaming Blocks API.

2.1 Application UI Overview

The application is a two-tab Gradio interface. The Analyse tab accepts a photograph via upload or webcam and streams five pipeline progress steps before showing results. The Supported Foods tab displays all 79 recognised dishes grouped by category.

Figure 2.1a *App UI — empty state*

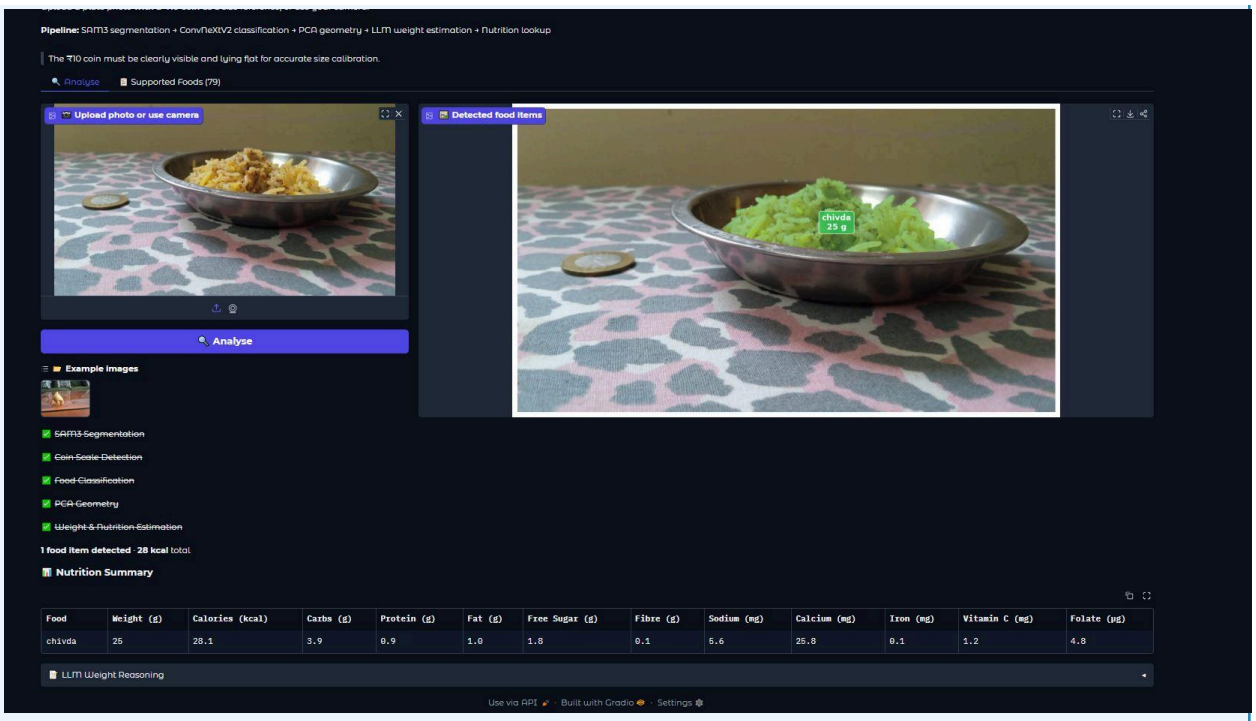


Figure 2.1b *App UI — successful detection of chivda (25 g, 28 kcal) with annotated output image*

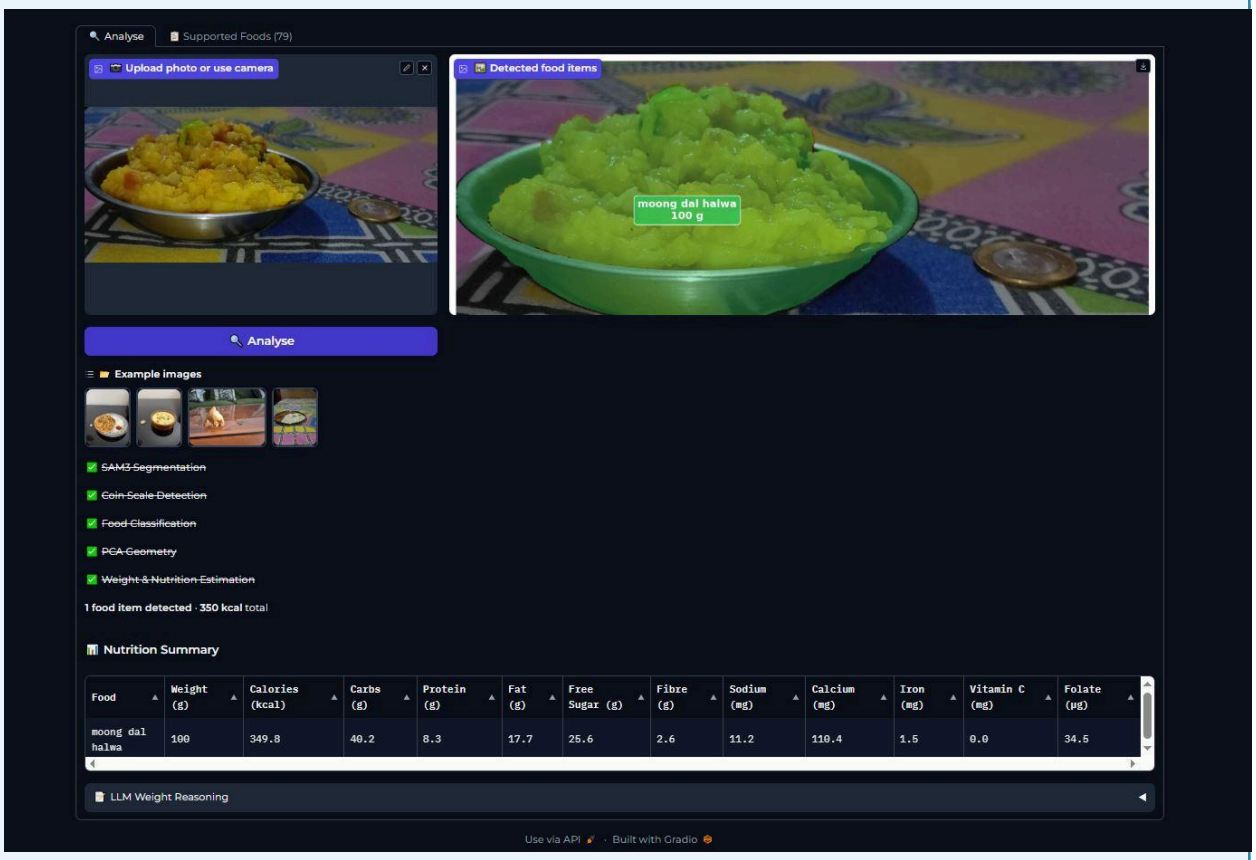


Figure 2.1c *App UI — successful detection of moong dal halwa (100 g, 350 kcal) with full nutrition table*

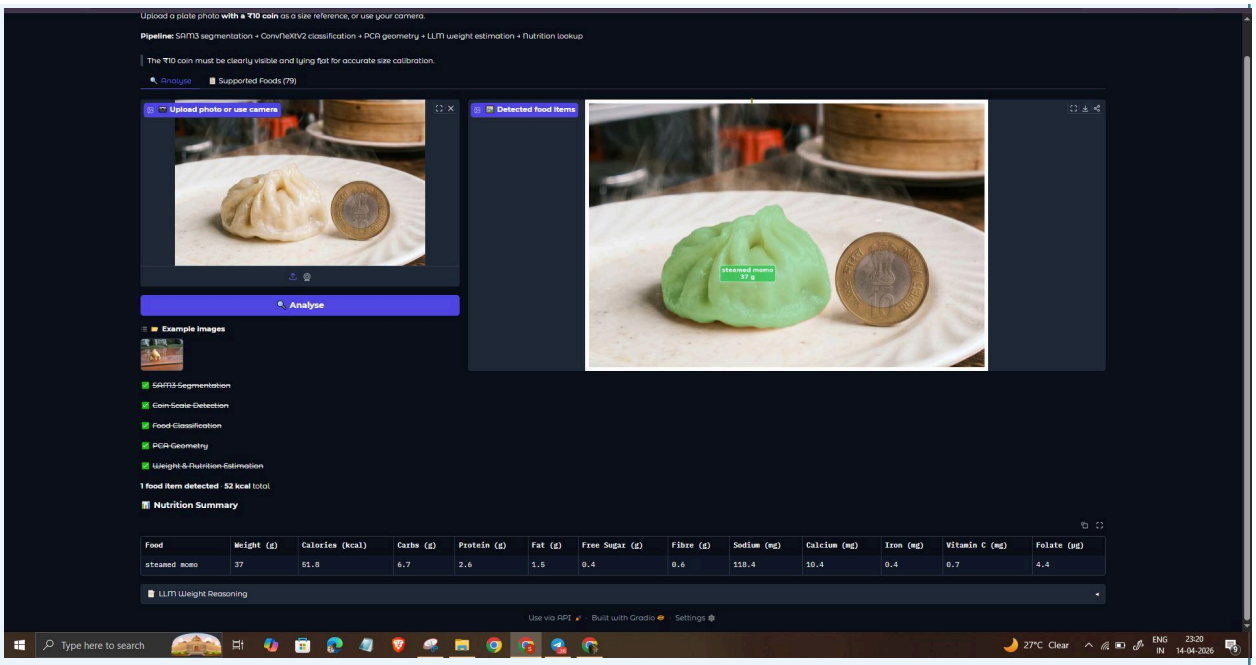


Figure 2.1d *App UI — steamed momo detection (37 g, 52 kcal) from stock image with ₹10 coin)*

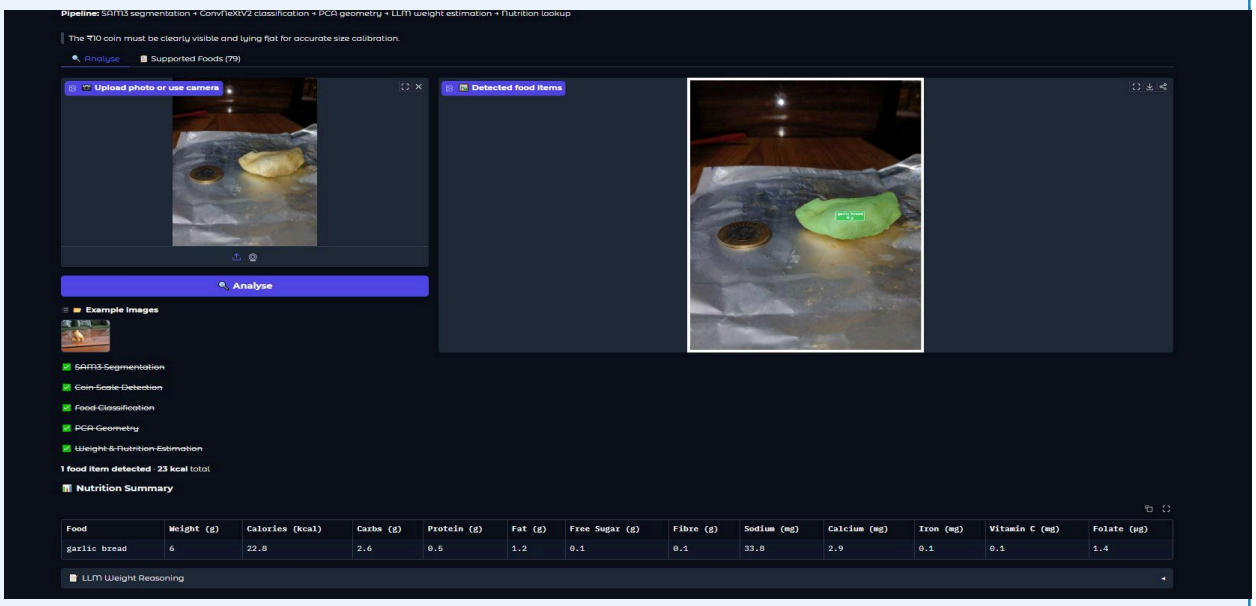


Figure 2.1e *App UI — garlic bread misclassification (momos image, detected as garlic bread 6 g)*

2.2 Result Interpretation Guide

UI Element	What it Shows
Progress checklist	Streaming 5-step pipeline status — SAM3 → Coin → Classify → PCA → Weight

Annotated output image	Original image with coloured SAM3 masks + food label + weight estimate overlaid
Nutrition Summary table	Calories, carbs, protein, fat, sugar, fibre, sodium, calcium, iron, vitamin C, folate for each detected item
LLM Weight Reasoning	Accordion showing geometry model (cylinder/hemisphere etc.), density used, and one-sentence reasoning per item
⚠️ No food items warning	ConvNeXtV2 confidence below 40% threshold for all segments — food not in supported list or poor image quality
❌ No coin detected	SAM3 could not find a ₹10 coin — pipeline halts at step 2 since coin is required for scale calibration

2.3 Sample Test Results from Real Images

The following results were obtained by running the live application on actual food photographs with a ₹10 coin placed in frame:

Food Image	Detected Class	Weight (g)	Calories (kcal)	All 5 Steps Passed?
Biryani in steel bowl + coin	chivda ⚠️	25	28.1	✅ Yes
Moong dal halwa in bowl + coin	moong dal halwa ✅	100	349.8	✅ Yes
Steamed momo (stock image) + coin	steamed momo ✅	37	51.8	✅ Yes
Samosa on foil + coin (small)	garlic bread ⚠️	6	22.8	✅ Yes (wrong class)

⚠️ **Note on chivda misclassification:** The biryani image was misclassified as chivda. This is expected — biryani in a steel bowl at distance has a grainy top surface visually similar to chivda. This reflects the fine-grained challenge of Indian food classification and the importance of image quality and angle. Same reason for steamed momos misclassification.

3. Codebase Structure & Design

3.1 File Architecture

File	Role
app.py	Gradio UI — 517 lines. Defines the two-tab interface, streaming callback, nutrition table builder, session directory management, and example image gallery.

pipeline.py	Core ML pipeline — 839 lines. All SAM3 segmentation, coin scale extraction, ConvNeXtV2 classification, PCA geometry, LLM weight estimation, and nutrition scaling logic.
models.py	Lazy model loader — globally caches SAM3 (facebook/sam3) and ConvNeXtV2 checkpoints. Loads once on first call; thread-safe for concurrent Gradio sessions.
config.py	Single source of truth for all paths, thresholds, prompt strings, and LLM settings. Never hard-coded in pipeline.py.
food_nutrition.json	Per-100g nutritional values for 80 Indian food classes — calories, carbs, protein, fat, free sugar, fibre, sodium, calcium, iron, vitamin C, folate.
food_density.json	Apparent bulk density (g/cm³) for all 80 food classes as-served, with source and basis. Used to hard-enforce density in weight estimation.
requirements.txt	torch, torchvision, timm, transformers, albumentations, opencv-python-headless, pillow, numpy, scikit-learn, langchain-core, langchain-openai, openai, pandas, matplotlib, gradio, pydantic, huggingface_hub, gdown

3.2 config.py — Configuration

All runtime parameters are centralised in config.py, making it easy to adjust thresholds without touching pipeline logic:

```
python — config.py
CKPT_PATH          = '.../best_convnextv2_tiny.pt'   # classifier checkpoint
NUTRITION_PATH     = '.../food_nutrition.json'
DENSITY_PATH       = '.../food_density.json'
CONVNEXT_CONF_THRESH = 0.40   # classifier min confidence
CONF_THRESH          = 0.60   # SAM3 food segment confidence
IOU_THRESH           = 0.40   # mask merge IoU threshold
CONT_THRESH          = 0.25   # containment threshold (food-only vs container)
COIN_DIAMETER_CM     = 2.7    # RBI ₹10 coin known diameter
COIN_CONF_THRESH     = 0.50   # SAM3 coin detection confidence
OLLAMA_MODEL         = 'qwen3.5:397b-cloud'  # LLM for weight reasoning
PROMPT_1  = 'food and its bowl'   # SAM3 prompt for container+food
PROMPT_2  = 'drink and its glass' # SAM3 prompt for beverages
PROMPT_3  = 'food'                # SAM3 food-only prompt
PROMPT_COIN = 'coin'              # SAM3 coin detection prompt
```

3.3 models.py — Lazy Model Loading

Both SAM3 and ConvNeXtV2 are heavy GPU models. models.py uses module-level globals with None-check guards to load each model exactly once on first use, then cache for all subsequent calls. This avoids reloading on every Gradio request.

```
python — models.py (key sections)
# SAM3 — facebook/sam3 from HuggingFace Hub
_sam_model = Sam3Model.from_pretrained('facebook/sam3').to(device)
_sam_processor = Sam3Processor.from_pretrained('facebook/sam3')

# ConvNeXtV2 — loaded from local checkpoint
```

```
ckpt = torch.load(CKPT_PATH, map_location=device, weights_only=False)
class_to_idx = ckpt['class_to_idx']          # class name → index
_idx_to_class = {v: k for k, v in class_to_idx.items()}
num_classes  = ckpt['cfg']['num_classes']     # 80
img_size     = ckpt['cfg']['img_size']        # 176
model_name   = ckpt['cfg']['model_name']      # convnextv2_tiny.fcmae_ft_in22k_in1k
_cls_model = timm.create_model(model_name, pretrained=False,
             num_classes=num_classes, drop_rate=0.0)
_cls_model.load_state_dict(ckpt['model_state'])

# Inference transforms (must match training exactly)
_inference_tf = A.Compose([
    A.Resize(int(img_size*1.14), int(img_size*1.14)),
    A.CenterCrop(img_size, img_size),
    A.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
    ToTensorV2(),
])
```

4. Pipeline Code Deep-Dive (pipeline.py)

The pipeline is a streaming generator (run_pipeline_steps) that yields progress after each of five stages. This section documents each stage with the actual implementation code.

4.1 Stage 1 — SAM3 Segmentation

Four parallel SAM3 text-prompt queries are run: 'food and its bowl', 'drink and its glass', 'food', and 'coin'. The image is first resized to SAM3's native 1024px maximum to prevent CUDA OOM on high-resolution phone photos.

```python — SAM3 segmentation
# Resize large uploads (phone photos 3000-4000px+ OOM on 4x SAM passes)
_SAM_MAX_DIM = 1024
def _resize_for_sam(image_pil):
    w, h = image_pil.size
    if max(w, h) <= _SAM_MAX_DIM: return image_pil
    scale = _SAM_MAX_DIM / max(w, h)
    return image_pil.resize((int(w*scale), int(h*scale)), Image.LANCZOS)

# Run 4 SAM3 queries; clear CUDA cache between each
det1 = get_dets(sam_image, PROMPT_1)            # food + container
torch.cuda.empty_cache()
det2 = get_dets(sam_image, PROMPT_2)            # drink + glass
torch.cuda.empty_cache()
det3 = get_dets(sam_image, PROMPT_3)            # food only
torch.cuda.empty_cache()
coin_dets = get_dets(sam_image, PROMPT_COIN, conf_thresh=COIN_CONF_THRESH)
```

After collection, merge_detections() de-duplicates overlapping masks using IoU clustering, preferring container-including masks (PROMPT_1/2) over food-only masks (PROMPT_3) to avoid double-counting. Masks are merged into groups by IoU >= 0.4; within each group, the highest-confidence container mask wins.

```python — merge_detections() logic summary
def merge_detections(det_p1, det_p2, det_p3, prompt_labels,
                     iou_threshold=0.30, containment_threshold=0.25):
    # 1. Collect all masks with their source prompt tag
    # 2. Build NxN IoU matrix
    # 3. Group overlapping masks by IoU threshold (graph clustering)
    # 4. Within each group: prefer container prompt (P1/P2) over food-only (P3)
    # 5. Filter food-only masks contained (>25%) within a container mask
    # Result: one mask per distinct food item, no duplicates
```

4.2 Stage 2 — Coin Scale Extraction

The ₹10 RBI coin (known diameter 2.7 cm) is used as a fixed reference object. PCA is applied to the coin's segmentation mask to find its major axis in pixels, then the pixel-per-centimetre scale factor is computed.

```python — coin scale & PCA geometry
def extract_coin_scale(coin_dets, img_np, save_dir) -> float:
    # Raise CoinNotFoundError if no coin detected or mask too small (<50 px)
    best_idx  = int(np.argmax(coin_dets.confidence))
```

```
    coin_mask = coin_dets.mask[best_idx]

    # PCA on coin mask pixels → major axis width in pixels
    _, (w_px, h_px), _, _ = pca_geometry(coin_mask)
    major_px  = w_px
    roundness = h_px / w_px   # warns if coin is tilted (<0.6)

    px_per_cm = major_px / COIN_DIAMETER_CM   # COIN_DIAMETER_CM = 2.7
    return px_per_cm

def pca_geometry(mask):
    ys, xs  = np.where(mask)
    coords  = np.stack([xs, ys], axis=1).astype(np.float32)
    center  = coords.mean(axis=0)
    pca     = skPCA(n_components=2).fit(coords)
    stds    = np.sqrt(pca.explained_variance_)
    width_px  = 4.0 * stds[0]   # 4-sigma span of major axis
    height_px = 4.0 * stds[1]   # 4-sigma span of minor axis
    angle_deg = np.degrees(np.arctan2(pca.components_[0][1],
                                      pca.components_[0][0]))
    return center, (width_px, height_px), angle_deg, pca.components_
```

4.3 Stage 3 — ConvNeXtV2 Food Classification

Each SAM3 food segment is cropped and classified by ConvNeXtV2-Tiny. The crop is masked before classification (background pixels set to neutral grey 128) so the classifier focuses on the food item and is not confused by adjacent dishes.

```
python — ConvNeXtV2 classification
def classify_crop(crop_rgb, conf_thresh=CONVNEXT_CONF_THRESH):
    cls_model, idx_to_class, inference_tf = get_classifier()
    # Apply inference transforms (Resize → CenterCrop → Normalize → ToTensor)
    tensor = inference_tf(image=crop_rgb)['image'].unsqueeze(0)
    tensor = tensor.to(device, memory_format=torch.channels_last)
    with torch.no_grad(), torch.amp.autocast('cuda'):
        probs = torch.softmax(cls_model(tensor), dim=1)[0]
    top_prob, top_idx = probs.max(0)
    conf = top_prob.item()
    if conf < conf_thresh:   # CONVNEXT_CONF_THRESH = 0.40
        return None, conf    # reject low-confidence predictions
    return idx_to_class[top_idx.item()], conf

# Before classification: mask background to neutral grey
masked_crop = original_crop.copy()
masked_crop[~crop_mask] = 128   # hide adjacent food items
```

4.4 Stage 4 — PCA Geometry

For each classified food segment, PCA extracts the rotation-invariant major and minor axes. The pixel measurements are converted to real-world centimetres using the coin calibration factor. A visualisation image (PCA ellipse + axis arrows) is saved and passed to the LLM.

```
python — PCA geometry stage
def _save_pca_image_for_llm(item, pixels_per_cm, seg_idx, save_dir):
    center, (width_px, height_px), angle_deg, comps = pca_geometry(crop_mask)
    width_cm  = round(width_px  / pixels_per_cm, 1)  # real-world width
    height_cm = round(height_px / pixels_per_cm, 1)  # real-world height
```

```
    # Create matplotlib figure with:
    #   - food crop image as background
    #   - blue overlay on segmented region
    #   - dashed PCA ellipse
    #   - red double-arrow for major axis (W)
    #   - green double-arrow for minor axis (H)
    #   - numeric labels W=X.X cm, H=Y.Y cm
    # Saved to pca_NN_foodname.png — sent to LLM only
```

4.5 Stage 5 — LLM Weight Estimation

The LLM receives the PCA visualisation image + a structured prompt with measured dimensions. Critically, density is never left to the LLM — it is looked up from food_density.json and hard-enforced by overwriting the LLM's output:

```python — LLM weight estimation with density enforcement
def estimate_weight(item) -> WeightEstimate:
    # 1. Look up density from static JSON — LLM cannot override this
    density_val, density_src = _get_density(item['food_class'])

    # 2. Build prompt with PCA image + measured dimensions
    prompt = (
        f'Food item: {item["food_class"]}\n'
        f'Step 1 — Measured dimensions (PCA + coin scale):\n'
        f'  • Major axis width  (W): {item["width_cm"]} cm\n'
        f'  • Minor axis height (H): {item["height_cm"]} cm\n'
        f'Step 2 — Visual reference: [PCA image attached]\n'
        f'Step 4 — Estimate weight. Choose a geometric model '
        f'(hemisphere/cylinder/sphere/flat disc), compute volume, '
        f'multiply by density.\n'
        + _parser.get_format_instructions()   # Pydantic JSON schema
    )

    # 3. Invoke multimodal LLM (image + text)
    msg = HumanMessage(content=[
        {'type': 'image_url', 'image_url': {'url': f'{b64}'}},
        {'type': 'text', 'text': prompt},
    ])
    result = _parser.invoke(_llm.invoke([msg]))

    # 4. HARD-ENFORCE density — LLM output is OVERWRITTEN
    result.density_g_per_cm3 = density_val   # from food_density.json
    result.density_source    = density_src
    return result
```

The WeightEstimate Pydantic model ensures structured output: weight_g (float), geometry (string), density_g_per_cm3 (float), reasoning (string). If the LLM response fails to parse, a graceful fallback returns 100 g with an error note, and the pipeline continues.

4.6 Nutrition Scaling

```python — nutrition scaling
def scale_nutrition(food_class, weight_g) -> dict:
    entry = _fuzzy_lookup(food_class)   # exact → normalised → substring match
    if entry is None: return None
    factor = weight_g / 100.0
```

```
    scaled = {'food_class': food_class, 'weight_g': round(weight_g, 1)}
    for col in NUTRIENT_COLS:           # 11 nutrients from config.py
        raw_val = entry.get(col)
        scaled[col] = round(raw_val * factor, 2) if raw_val else None
    return scaled

# _fuzzy_lookup: exact key match → strip punctuation → substring match
# Handles minor spelling differences between classifier and DB keys
```

4.7 Annotated Output Image

The combined annotated image overlays coloured SAM3 masks + food label + weight (in grams) for each detected item onto the original photograph, using matplotlib's OO API (Figure/FigureCanvasAgg) for thread safety:

```
python — annotated output image generation
def save_combined_annotated_image(image_pil, final_results, full_masks, save_dir):
    fig = Figure(figsize=(max(8, W/100), max(6, H/100)), dpi=120)
    FigureCanvasAgg(fig)   # thread-safe — no pyplot global state
    ax = fig.add_subplot(111)
    ax.imshow(img_np)
    for idx, (res, mask) in enumerate(zip(final_results, full_masks)):
        color = _PALETTE[idx % len(_PALETTE)]   # 8-colour rotating palette
        ov = np.zeros((H, W, 4), dtype=np.float32)
        ov[mask] = [*color, 0.38]   # semi-transparent colour overlay
        ax.imshow(ov)
        cx, cy = xs.mean(), ys.mean()   # mask centroid for label placement
        ax.text(cx, cy, f"{res['food_class']}\n{res['weight_g']:.0f} g",
                ha='center', va='center', fontweight='bold', color='white',
                bbox=dict(boxstyle='round,pad=0.35', facecolor=color, alpha=0.82))
    fig.savefig(os.path.join(save_dir, 'annotated_result.png'), dpi=120)
```

5. Gradio Application Architecture (app.py)

5.1 Streaming Generator Pattern

The key design decision is that process_image() is a Python generator, enabling Gradio to stream incremental updates to the UI as each pipeline stage completes. Without demo.queue(), Gradio would buffer all yields and only render the final state.

```
python — streaming generator pattern
def process_image(image, session_dir):
    # Generator — yields (progress_md, annotated_img, table_df, reasoning, dir)
    for step_idx, _label, partial in run_pipeline_steps(pil_img, new_dir):
        if partial is None:
            # Mid-pipeline: show step progress only
            yield (_progress_md(step_idx), None, pd.DataFrame(), '', new_dir)
        else:
            # Final step: show results or error
            final_results, annotated_path, error = partial
            ...
            yield (progress_md + summary, annotated_img, table_df, reasoning,
new_dir)

# CRITICAL: queue() must be called before launch() for streaming to work
demo.queue()
demo.launch(server_name='0.0.0.0', server_port=7860, share=True)
```

5.2 Session Management

Each Gradio user session gets its own isolated temporary directory for intermediate files (PCA images, coin debug, annotated result). Old directories are pruned every hour by a daemon thread to prevent disk accumulation on the server.

```
python — session management
def _new_session_dir() -> str:
    return tempfile.mkdtemp(prefix='food_nutr_')   # unique per request

# Background daemon thread — prunes dirs older than 1 hour
def _prune_old_temp_dirs():
    while True:
        for name in os.listdir(temp_base):
            if name.startswith('food_nutr_') and mtime > 3600:
                shutil.rmtree(path, ignore_errors=True)
        time.sleep(3600)

threading.Thread(target=_prune_old_temp_dirs, daemon=True).start()
```

5.3 Progress Markdown Builder

The five-step checklist uses strikethrough for completed steps, bold for the current step, and plain text for pending steps — rendered live in the Gradio Markdown component:

```
python — progress markdown
def _progress_md(current_step, done=False, error=False) -> str:
    for i, label in enumerate(_STEP_LABELS, start=1):
```

```
        if done or i < current_step:  lines.append(f'✅  ~~{label}~~')
        elif i == current_step:        lines.append(f'❌  **{label}**' if error
                                                    else f'⏳  **{label}** …')
        else:                          lines.append(f'⬜  {label}')
```

5.4 Supported Foods Tab

The Supported Foods tab is rendered as custom HTML with CSS pill badges grouped by category (Breads & Rice, Snacks & Chaat, Curries & Mains, Sweets & Desserts, Pizza, Drinks). The HTML is generated programmatically from the _CATEGORY_MAP dictionary.

6. Data Files — food_nutrition.json & food_density.json

6.1 food_nutrition.json Structure

Contains per-100g nutritional values for all 80 supported Indian foods. Each entry has 11 nutrient fields:

```
json — food_nutrition.json structure
{ "nutrition": {
    "aloo paratha": {
      "calories_kcal": 200.0,
      "carbohydrates_g": 29.0,
      "protein_g": 5.5,
      "fats_g": 7.3,
      "free_sugar_g": 0.5,
      "fibre_g": 2.0,
      "sodium_mg": 300.0,
      "calcium_mg": 60.0,
      "iron_mg": 1.8,
      "vitamin_c_mg": 5.0,
      "folate_ug": 30.0
    }, ...
  }
}
```

6.2 food_density.json Structure

Contains the apparent bulk density (g/cm³) of each food as served, along with the scientific basis and source. This is the single authoritative density value used in weight estimation — the LLM output is overwritten with this value.

Food	Density (g/cm³)	Basis	Source / Rationale
idli	0.53	published	PMC/Springer study — cooked idli bulk density 0.53 g/cm³
gulab jamun	1.12	inference	Fried khoya ball + ~30% sugar syrup absorption → >water density
jalebi	1.25	inference	Fried wheat batter coil + heavy sugar-syrup saturation
kaju katli	1.38	inference	Cashew paste + sugar; compressed flat diamond, near-zero air
poha	0.38	inference	Flattened puffed rice flakes — one of the lightest cooked foods
biryani	0.88	FAO-inference	FAO cooked rice ~1.0; biryani loosely layered with meat/veg → ~0.88

Density ranges from 0.28 g/cm³ (chivda — very high air content) to 1.38 g/cm³ (kaju katli — dense compressed candy). The distinction between 'published' values (e.g. idli from PMC) and 'inference' values (food engineering first-principles) is tracked per entry.

7. Datasets

NutriVision used two distinct data tracks aligned to the evolution of its pipeline: a detection track (early milestones) and a classification track (final pipeline). A custom nutrition database and a small in-house weight-estimation dataset supplemented both tracks.

7.1 Food Classification Dataset — Khana Dataset

The Khana dataset is a large-scale benchmark specifically designed for Indian food recognition. It formed the primary training corpus for the final classification backbone.

Attribute	Detail
Total images	131,819 validated food images
Food classes	80 Indian food categories
Train / Val / Test split	80% / 10% / 10% stratified (seed = 42)
Class imbalance	Min 12 images / class; Max 788 images / class; Avg ~164.8
Image diversity	Variable lighting, camera angle, plating style, backgrounds
Purpose	Training and evaluation of EfficientNetV2-S and ConvNeXtV2-Tiny classifiers

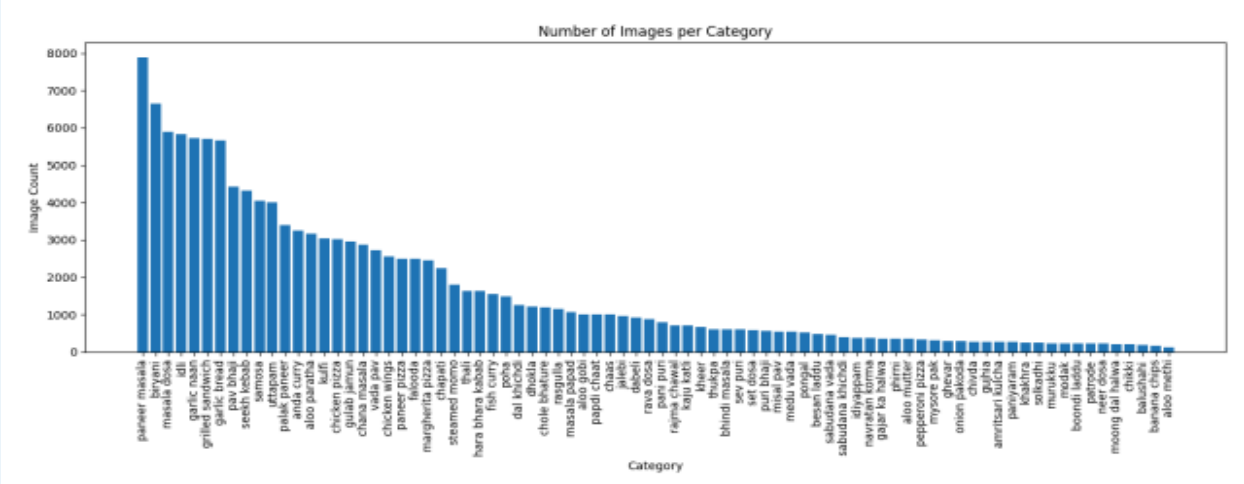


Figure 1: *Khana Dataset — class distribution bar chart (80 food categories)*

7.2 Detection Datasets (Earlier Pipeline — Milestones 2 & 3)

Three datasets were merged to train the YOLO-based detection pipeline in early milestones:

Dataset	Images	Classes	Key Features

IndianFood10 (Roboflow)	11,547	10	842 multi-dish plate images; YOLO segmentation masks
IndianFoodNet-30 (Roboflow)	5,446	30	Biryani, dosa, butter chicken; bounding box format
South Indian Food Detection	9,338	~31	Geographic balance for south-Indian cuisine
Indian Food YOLOv5 (M3 addition)	~14,000	81	Region-specific dishes; merged to 120 unified classes
TOTAL (merged, M3)	**39,185**	**118**	After removing classes with <50 annotations

7.3 Nutrition Database

The Indian Food Nutritional Values dataset (Kaggle 2025) contains per-100g macro and micronutrient data for 310+ Indian dishes (calories, protein, carbs, fat, fiber, sodium, iron). Missing values were imputed with the class median. This static CSV underpins the nutrition retrieval stage throughout all milestones.

7.4 Weight Estimation Dataset (Custom)

A small dataset of approximately 88 overhead plate photographs was collected by the team using a phone camera and digital kitchen scale. Each image was labelled with food name, pixel area, pixel ratio (food/plate), and actual weight in grams. After quality filtering, 63 usable records remained. This data was used to train an MLP regression model in the early pipeline before being superseded by the PCA + density estimation approach in the final pipeline.

8. Data Preprocessing & Exploratory Data Analysis

8.1 Detection Dataset Preprocessing (Milestones 2 & 3)

Merging three datasets with different naming conventions required a canonical label-mapping strategy:

- Spelling normalisation: e.g. 'briyani' → 'biryani', 'Idli' → 'idli'.
- Format normalisation: e.g. 'PalakPaneer' → 'palak_paneer'.
- Regional synonyms collapsed: e.g. 'satham' → 'white_rice'.
- Class-index remapping: after sorting unified labels, each dataset's integer class IDs were re-mapped to the new unified index space.
- Low-frequency class removal: classes with fewer than 50 annotations were dropped, reducing the class count from 120 to 118.

- All images standardised to 640×640 pixels for YOLO training.

```
/content/nutrivision_merged/
│
├── data.yaml                    ← YOLO config
│
├── train/
│   ├── images/
│   │   ├── ds1_img1.jpg
│   │   ├── ds2_img45.png
│   │   └── ...
│   │
│   ├── labels/
│   │   ├── ds1_img1.txt
│   │   ├── ds2_img45.txt
│   │   └── ...
│
├── valid/
│   ├── images/
│   ├── labels/
│
├── test/
│   ├── images/
│   ├── labels/
```

Figure 2: *YOLO directory structure after merging all three datasets*

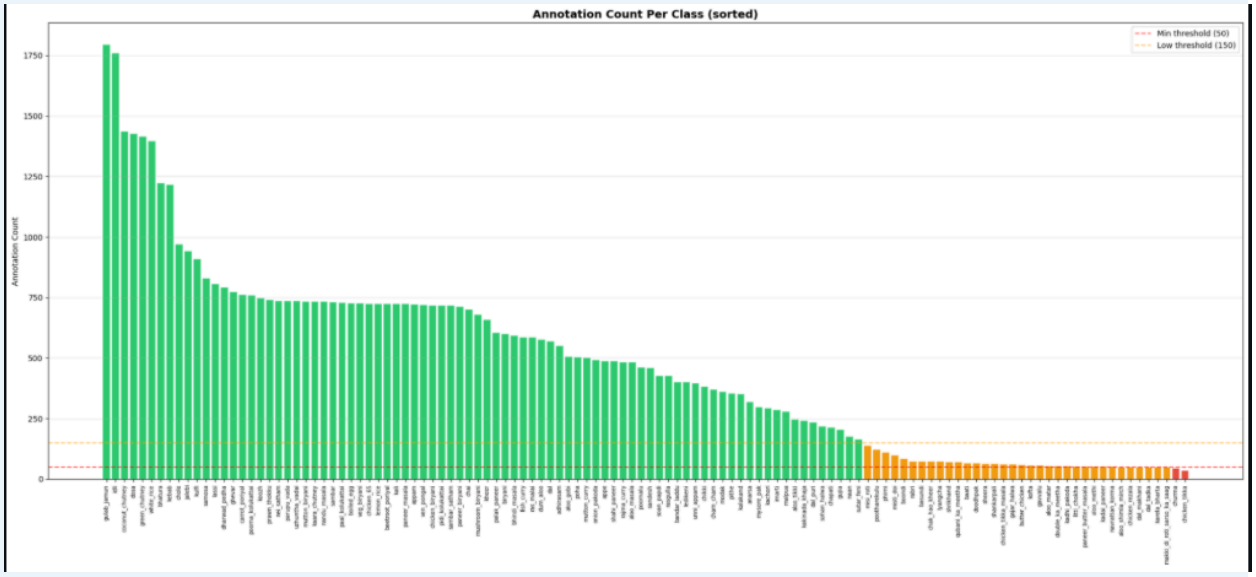


Figure 3: *Class distribution plot — 118 unified food classes with annotation counts*

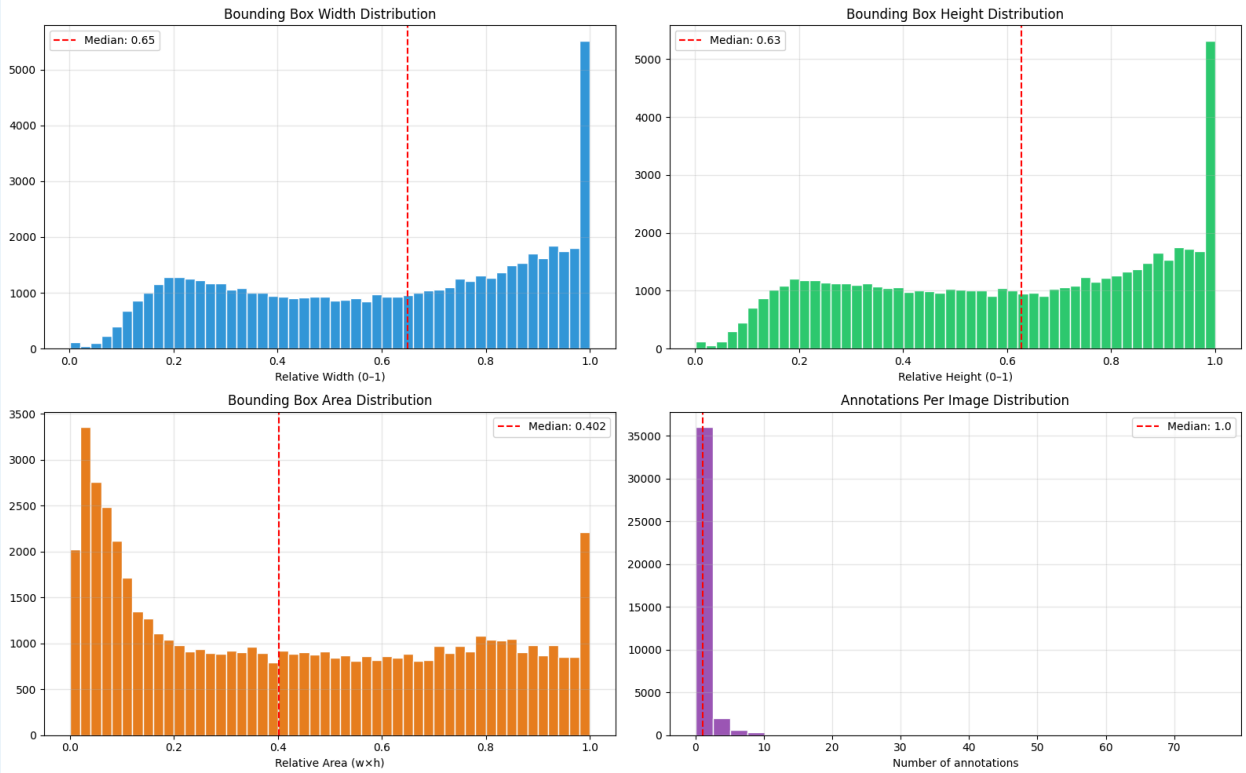


Figure 4: *Bounding box visualisation quality check on merged dataset*

8.2 Classification Dataset Preprocessing (Milestones 4 & 5)

For the Khana dataset, preprocessing was model-specific to match each architecture's training configuration:

Step	EfficientNetV2-S	ConvNeXtV2-Tiny
Resize	321 px → 300×300 centre crop	200 px → 176×176 centre crop
Normalisation (mean)	[0.5, 0.5, 0.5]	[0.485, 0.456, 0.406] (ImageNet)
Normalisation (std)	[0.5, 0.5, 0.5]	[0.229, 0.224, 0.225] (ImageNet)
Training augmentation	MixUp, CutMix, RandomResizedCrop, horizontal flip	RandomResizedCrop (176), horizontal flip, albumentations medium strength
Evaluation augmentation	None — centre crop only	None — centre crop only
Class imbalance handling	Loss-based weight adjustment	Weighted random sampler

Note: using the wrong image size or normalisation statistics during evaluation causes severe accuracy degradation — this was identified as the root cause of the initially poor ConvNeXtV2-Tiny results in Milestone 5, which were corrected by enforcing per-model preprocessing.

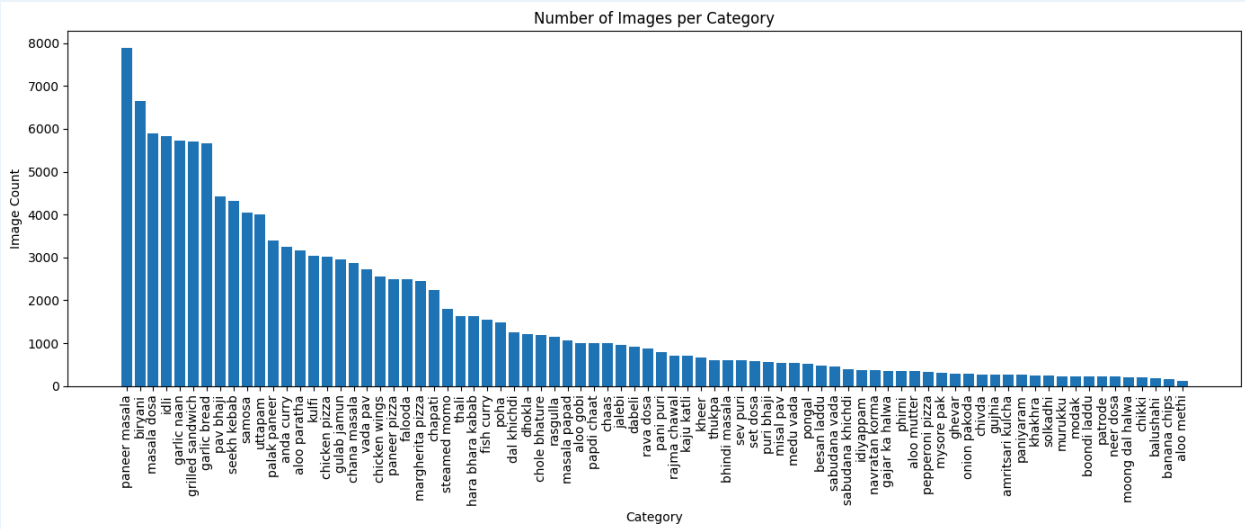


Figure 5: *Khana dataset EDA — image samples per class*

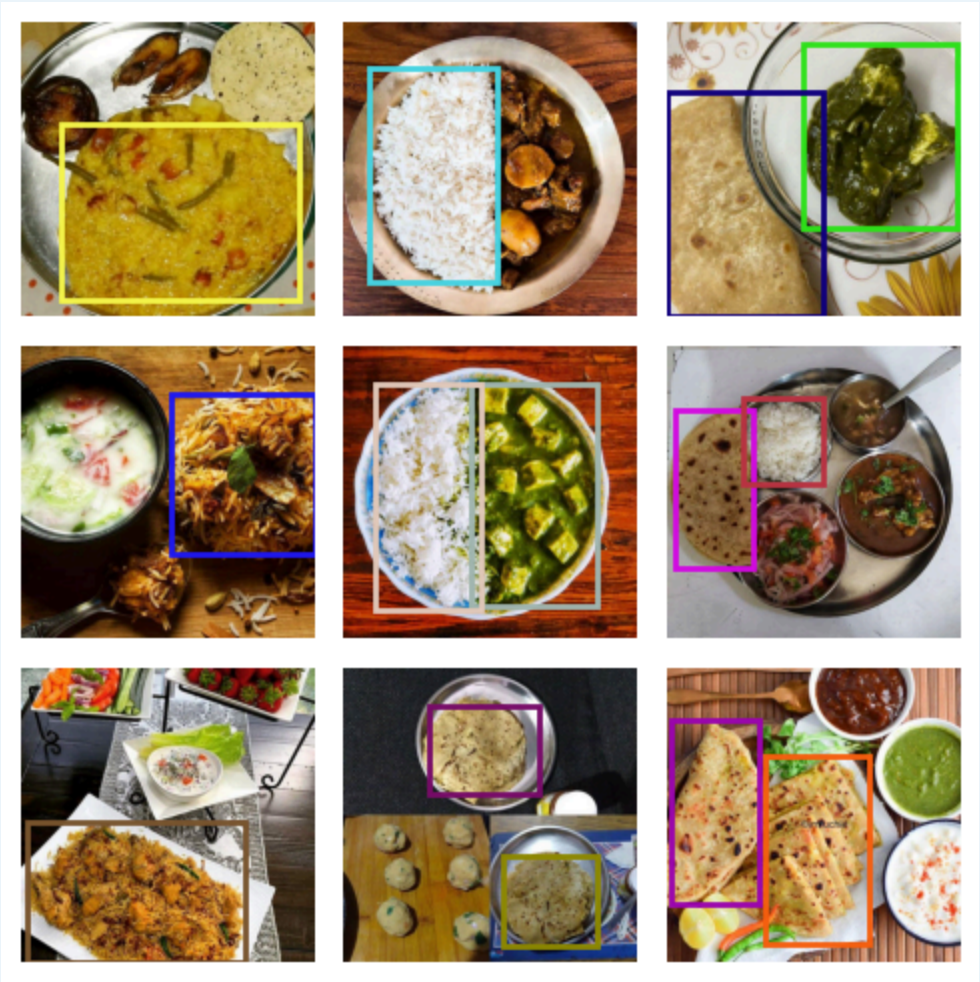


Figure 6: *Image dataset*

8.3 Nutrition Dataset EDA

The nutrition dataset was analysed across 310+ Indian dishes. Key findings: calorie distribution is right-skewed (most foods 50–250 kcal/100g); deep-fried snacks peak at 372 kcal; liquid dishes are lowest at 40–130 kcal. Macronutrient correlations confirmed fat is the dominant calorie driver, with protein weakly correlated to calories.

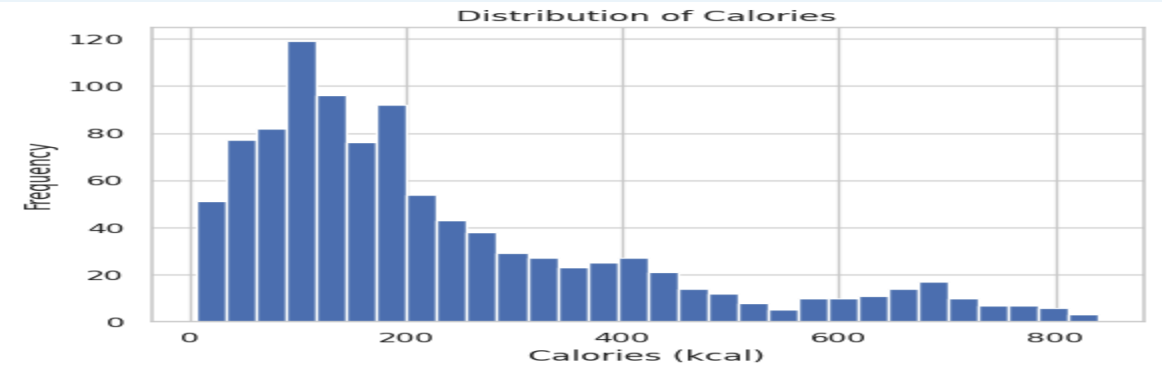
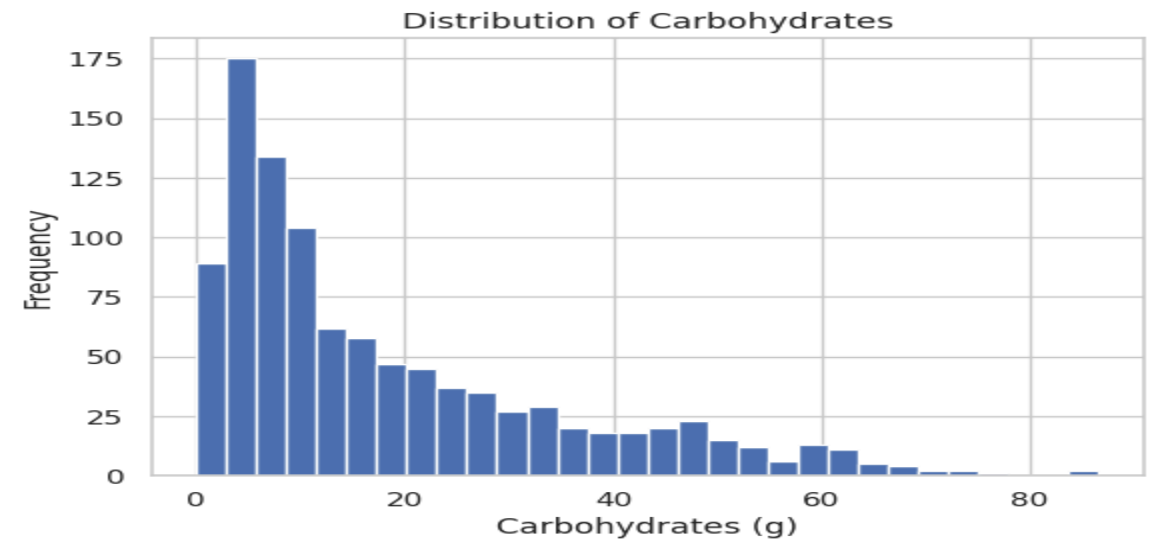
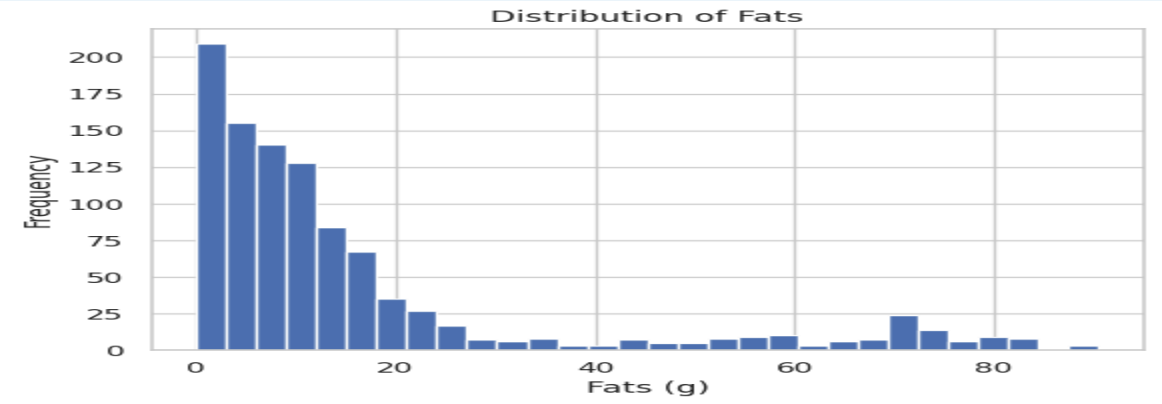


Figure 7: *Calorie distribution histogram across 310+ Indian foods*





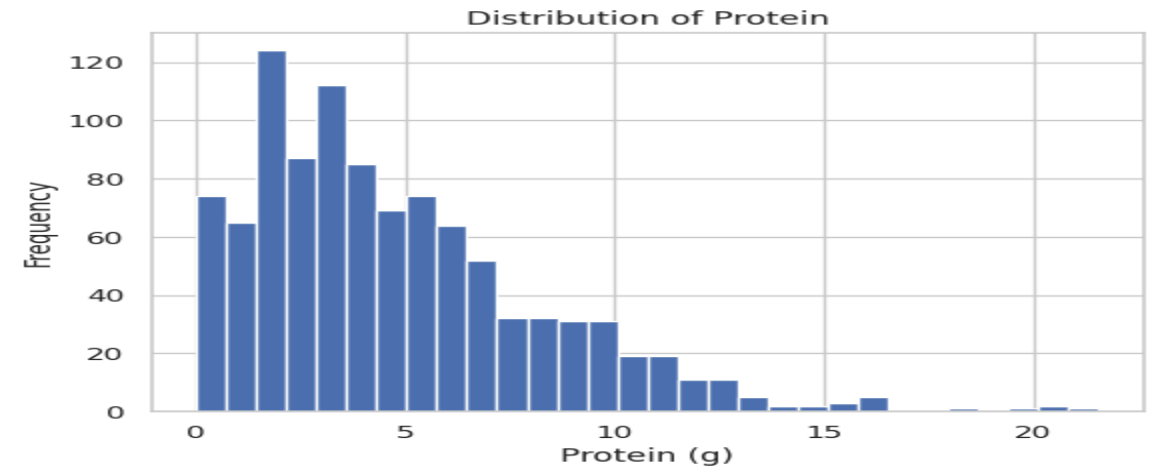


Figure 8: *Macronutrient profile by food category (protein/carbs/fat bar chart)* ▸ Insert from: **Nutrition.ipynb**
[Category macronutrient comparison cell]

9. Pipeline Evolution — From Detection to Segmentation-First

The NutriVision pipeline underwent a significant architectural evolution across milestones. Understanding this evolution is essential to understanding the final design choices.

9.1 Original Pipeline (Milestones 1 – 3)

Stage	Component	Description
1 — Detection	YOLOv8 / YOLOv12 / YOLOv26	Detect food items + bounding boxes
2 — Segmentation	SAM-2 (prompted by YOLO boxes)	Generate pixel masks per bounding box
3 — Weight Estimation	MLP regression model	pixel_ratio + pixel_food + class_encoded → weight (g)
4 — Nutrition Lookup	CSV + RAG (Pinecone)	Scale per-100g values by estimated weight
5 — Suggestions	Gemini API + MedGemma	RAG-grounded dietary recommendations

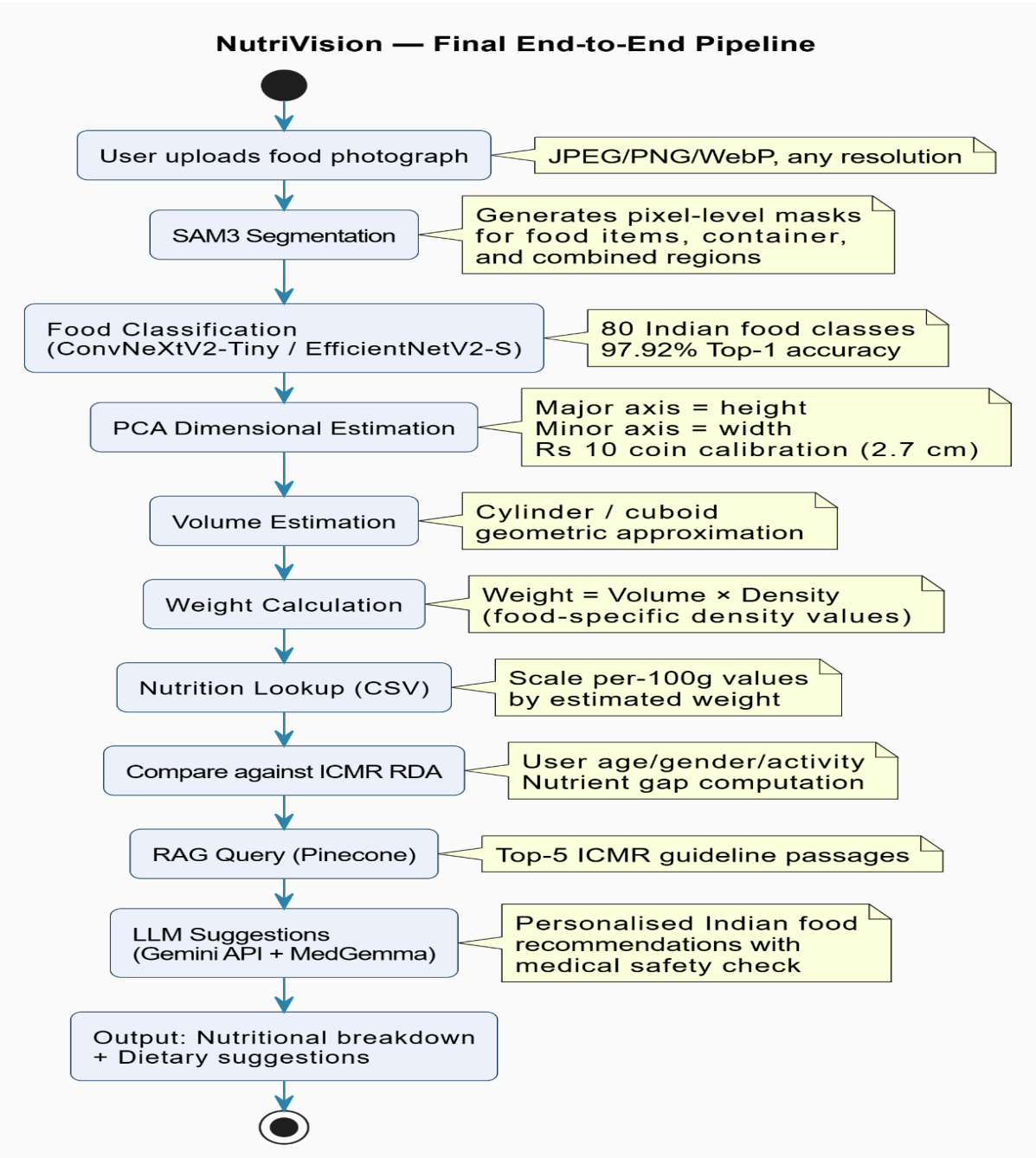
9.2 Revised Final Pipeline (Milestones 4 – 5)

The detection-first approach was replaced with a segmentation-first architecture using SAM3, and a dedicated classification model replaced the YOLO classification head. Weight estimation was redesigned from MLP regression to PCA-based geometric estimation with coin calibration.

Stage	Component	Description
1 — Segmentation	SAM3 (segment anything)	Direct pixel masks — food, container, food+container regions
2 — Classification	ConvNeXtV2-Tiny / EfficientNetV2-S	Classify each segmented food region into 80 Indian food classes
3 — Dimensional Estimation	PCA + coin calibration	Extract height/width from mask pixels; calibrate with Rs 10 coin (2.7 cm)
4 — Volume & Weight	Geometric approximation + density	Cylinder/cuboid approximation; multiply by food-specific density
5 — Nutrition Lookup	Structured CSV	Scale per-100g nutrient values by estimated weight

6 — Recommendations	MedGemma + Gemini API	ICMR RDA-grounded personalised dietary suggestions

9.3 Key Reasons for Pipeline Redesign



- SAM3 eliminates dependency on accurate bounding boxes and handles overlapping multi-dish scenarios more naturally.
- Dedicated classification models (ConvNeXtV2, EfficientNetV2) significantly outperform YOLO classification heads on fine-grained food categories.
- PCA-based weight estimation is more interpretable and does not require the large labelled regression dataset that the MLP approach demanded.
- Coin calibration provides a practical pixel-to-cm scale factor without requiring depth sensors.

10. Model Architecture & Training

10.1 Food Classification Models

Two deep learning architectures were selected as the primary food classifiers. Both were fine-tuned on the Khana dataset's 80 Indian food classes using pretrained ImageNet weights.

10.1.1 ConvNeXtV2-Tiny

ConvNeXtV2-Tiny is a modernised pure-convolutional network incorporating FCMAE (Fully Convolutional Masked AutoEncoder) pretraining. It was identified as the strongest single model in the project.

Parameter	Value
Parameters	27.9 M
Input image size	176 × 176 px
Batch size	128 (baseline); 256 via gradient accumulation (best)
Epochs	5 (baseline); 1 fine-tuning epoch with best hyperparameters
Optimiser	AdamW — Head LR: 1e-3, Backbone LR: 1e-4, Weight decay: 1e-2
Loss function	Cross-Entropy with label smoothing ε = 0.1
LR scheduler	1-epoch warmup → cosine decay
Regularisation	Dropout 0.2, weight decay, early stopping (patience=3)
Mixed precision	Yes (torch.amp.autocast)
Normalisation	ImageNet mean/std [0.485,0.456,0.406] / [0.229,0.224,0.225]
Test Top-1 Accuracy	97.92% (corrected evaluation)
Test Macro F1	0.9746
Inference throughput	150.7 images/sec (Tesla T4)

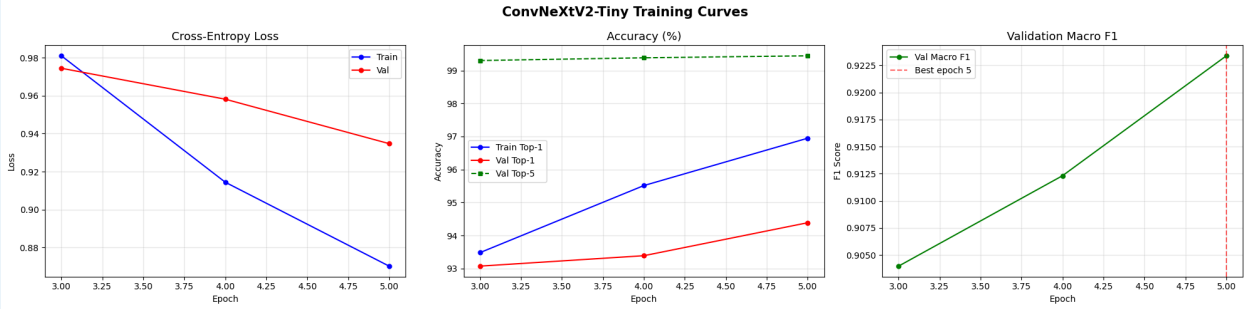


Figure 9: *ConvNeXtV2-Tiny training curves — Loss, Accuracy, and Val Macro F1 per epoch*

10.1.2 EfficientNetV2-S

EfficientNetV2-S provides a strong balance between accuracy and inference speed. It was trained using a two-phase fine-tuning strategy.

Parameter	Value
Parameters	20.3 M (tf_efficientnetv2_s)
Input image size	300 × 300 px
Phase 1 (head-only)	3 epochs — backbone frozen; head LR 1e-3
Phase 2 (full fine-tune)	7 epochs — backbone LR 1e-5, classifier LR 1e-4
Augmentation	MixUp, CutMix (Phase 2); standard transforms (Phase 1)
Normalisation	Mean/std = [0.5, 0.5, 0.5]
Test Top-1 Accuracy	92.85%
Test Macro F1	0.9024
Inference throughput	109.2 images/sec (Tesla T4)

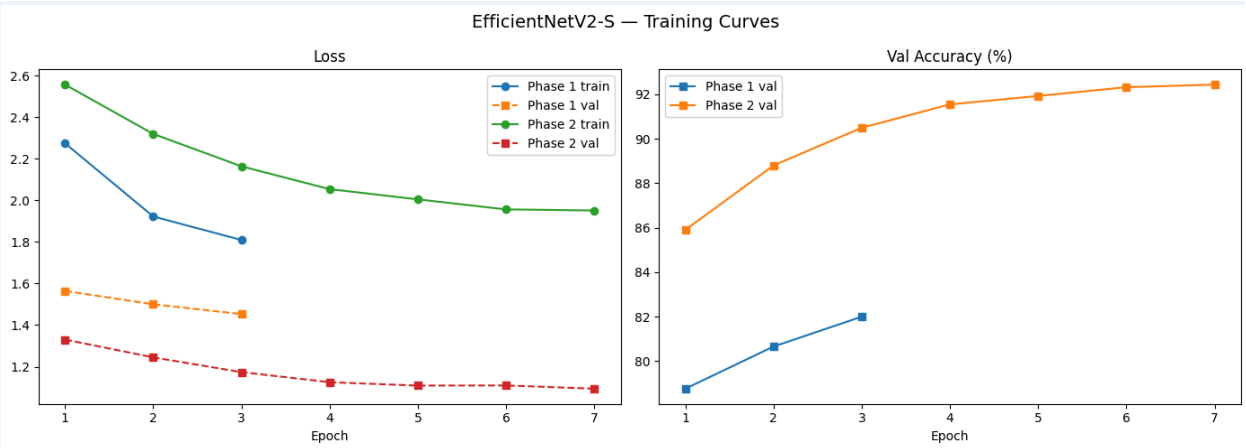


Figure 10: *EfficientNetV2-S training curves — Phase 1 & Phase 2 loss and validation accuracy*

10.2 YOLO Detection Models (Earlier Pipeline)

Two YOLO variants were trained in Milestones 3–4 on the merged detection dataset (118 classes, 39,185 images):

Model	Epochs	Batch	Precision	Recall	mAP@50	mAP@50-95
YOLOv26n	30	8	0.807	0.768	0.840	0.672
YOLOv12s	50	32	0.891	0.877	~0.90	~0.75

YOLOv12s used AdamW optimiser, Mosaic + HSV jitter + random erasing augmentation, and mixed precision training on a Tesla T4 GPU. Both models were superseded by the segmentation-first pipeline in the final architecture.

10.3 MobileNetV2 (Baseline Reference)

MobileNetV2 was trained as a lightweight baseline for initial validation of the classification approach. With a 128×128 input, 10 epochs, Adam optimiser, and CPU training, it achieved 72.06% accuracy and 0.6883 F1-score — sufficient to validate the approach but significantly below the final models.

11. Weight Estimation — From MLP to PCA + Geometry

11.1 Why Depth Estimation Was Rejected

Approach	Suitable?	Reason
ZoeDepth / MiDaS (3-D depth)	No	Dal, sambar, curry are liquid — depth map unreliable
Volume reconstruction	No	Roti/dosa are flat; near-zero height gives incorrect volume
Pixel ratio → MLP	Partial	Camera-distance-dependent; requires labelled regression data
PCA + coin calibration (chosen)	Yes	Geometric, interpretable, no specialised hardware, works for flat and 3-D food shapes

11.2 MLP Regression Approach (Milestones 2–3)

A shallow MLP was trained on 63 custom-photographed plate images. Features: pixel_ratio (food area / plate area), pixel_food, pixel_plate, class_encoded. The MLP directly predicted weight in grams. This approach was abandoned due to its small training set and sensitivity to camera distance.

11.3 Final PCA-Based Geometric Estimation (Milestones 4–5)

The PCA module applies Principal Component Analysis to the foreground pixels of each SAM3 segmentation mask to extract the major and minor axes, providing rotation-invariant estimates of food item height and width.

- PCA identifies the principal axes of the pixel distribution, yielding major axis (height) and minor axis (width).
- A Rs 10 coin (known diameter: 2.7 cm) serves as the calibration reference — detected in the image to compute pixels-per-cm.
- Real-world dimensions are computed by applying the pixel-to-cm scale factor to PCA outputs.
- Volume is estimated by geometric approximation (cylinder for round items, cuboid for flat breads) with assumed thickness per food category.
- Weight (g) = Volume (cm³) × food-specific density (g/cm³) from public references.

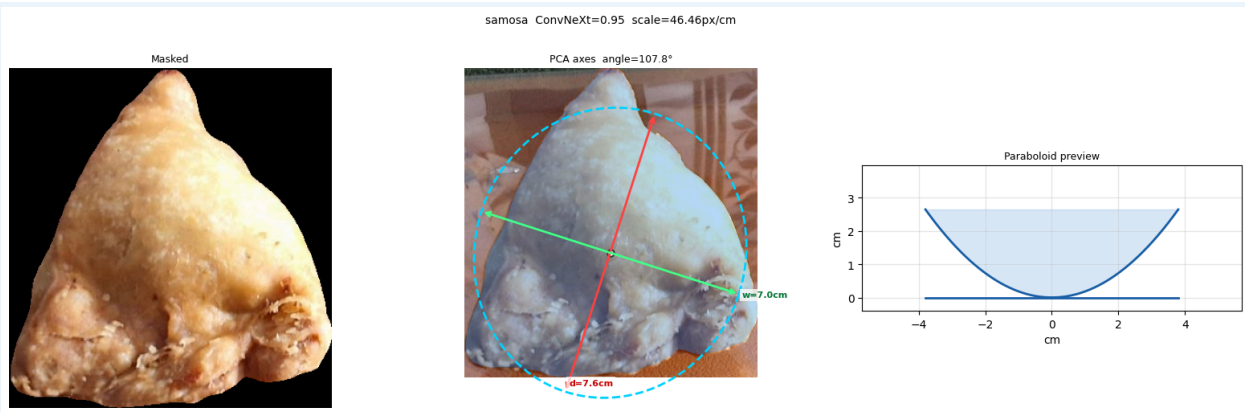
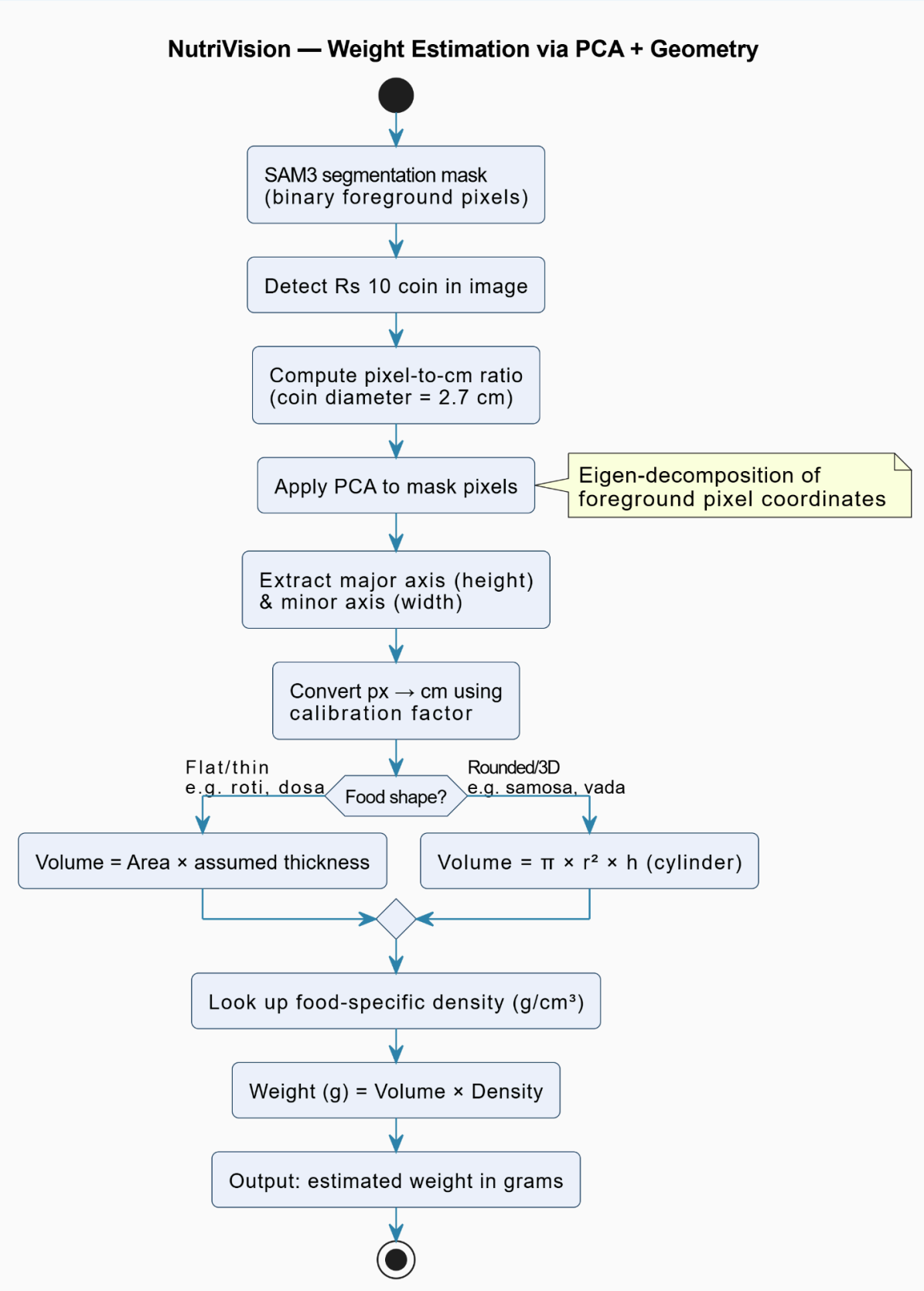


Figure 11: *PCA output on samosa — principal axes overlaid on segmentation mask, plus parabolic profile curve*


NutriVision — Weight Estimation via PCA + Geometry
SAM3 segmentation mask (binary foreground pixels)
Detect Rs 10 coin in image
Compute pixel-to-cm ratio (coin diameter = 2.7 cm)
Apply PCA to mask pixels
Eigen-decomposition of foreground pixel coordinates
Extract major axis (height) & minor axis (width)
Convert px → cm using calibration factor
Flat/thin e.g. roti, dosa
Food shape?
Rounded/3D e.g. samosa, vada
Volume = Area × assumed thickness
Volume = π × r² × h (cylinder)
Look up food-specific density (g/cm³)
Weight (g) = Volume × Density
Output: estimated weight in grams

12. Model Evaluation & Results

All classification models were evaluated on the stratified 10% hold-out test set (13,182 images) using standardised per-model preprocessing. Evaluation was performed in a reproducible Colab notebook with a fixed random seed of 42.

12.1 Classification Results Summary

Model	Top-1 Acc	Top-5 Acc	Precision	Recall	Macro F1	mAP
EfficientNetV2-S	92.85 %	99.12%	0.9251	0.8877	0.9024	0.9387
ConvNeXtV2-Tiny	97.92 %	99.76%	0.9746	0.9753	0.9746	0.9859
Ensemble (avg)	**98.00 %**	99.75%	0.9771	0.9757	0.9761	**0.9860**

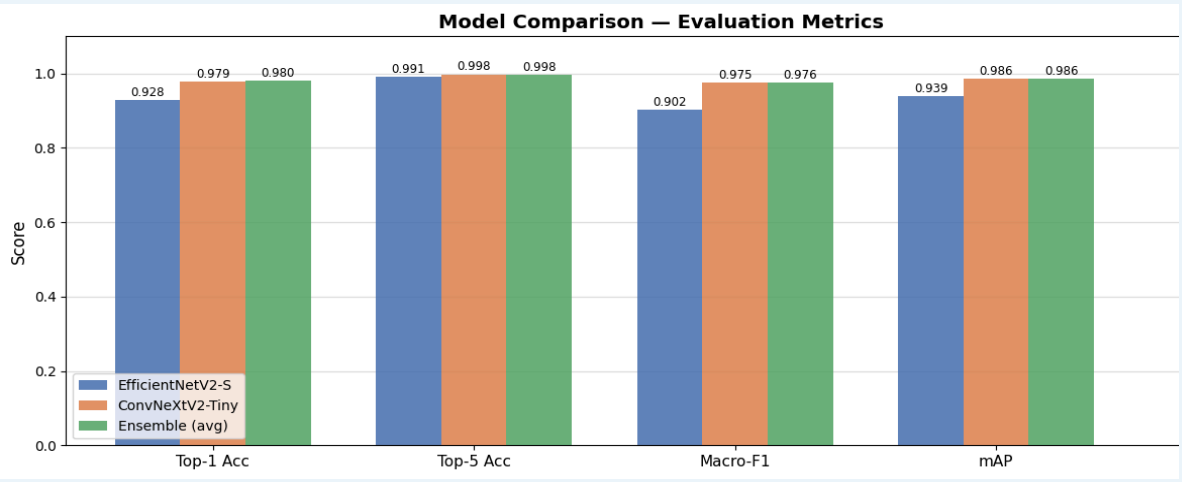


Figure 12: *Model Comparison Bar Chart — Top-1, Top-5, Macro-F1, mAP for all three configurations*

12.2 Confusion Matrix Analysis

Both models produce a strong diagonal pattern in their 80-class confusion matrices, indicating accurate predictions across the majority of categories. The dominant confusion pattern in both models is between visually similar pizza variants:

Confused Pair	EfficientNetV2-S	ConvNeXtV2-Tiny
Paneer pizza → Chicken pizza	48 cases	24 cases
Chicken pizza → Paneer pizza	38 cases	11 cases
Chicken pizza → Margherita pizza	20 cases	9 cases
Phirni / Kheer	Some cases	Negligible

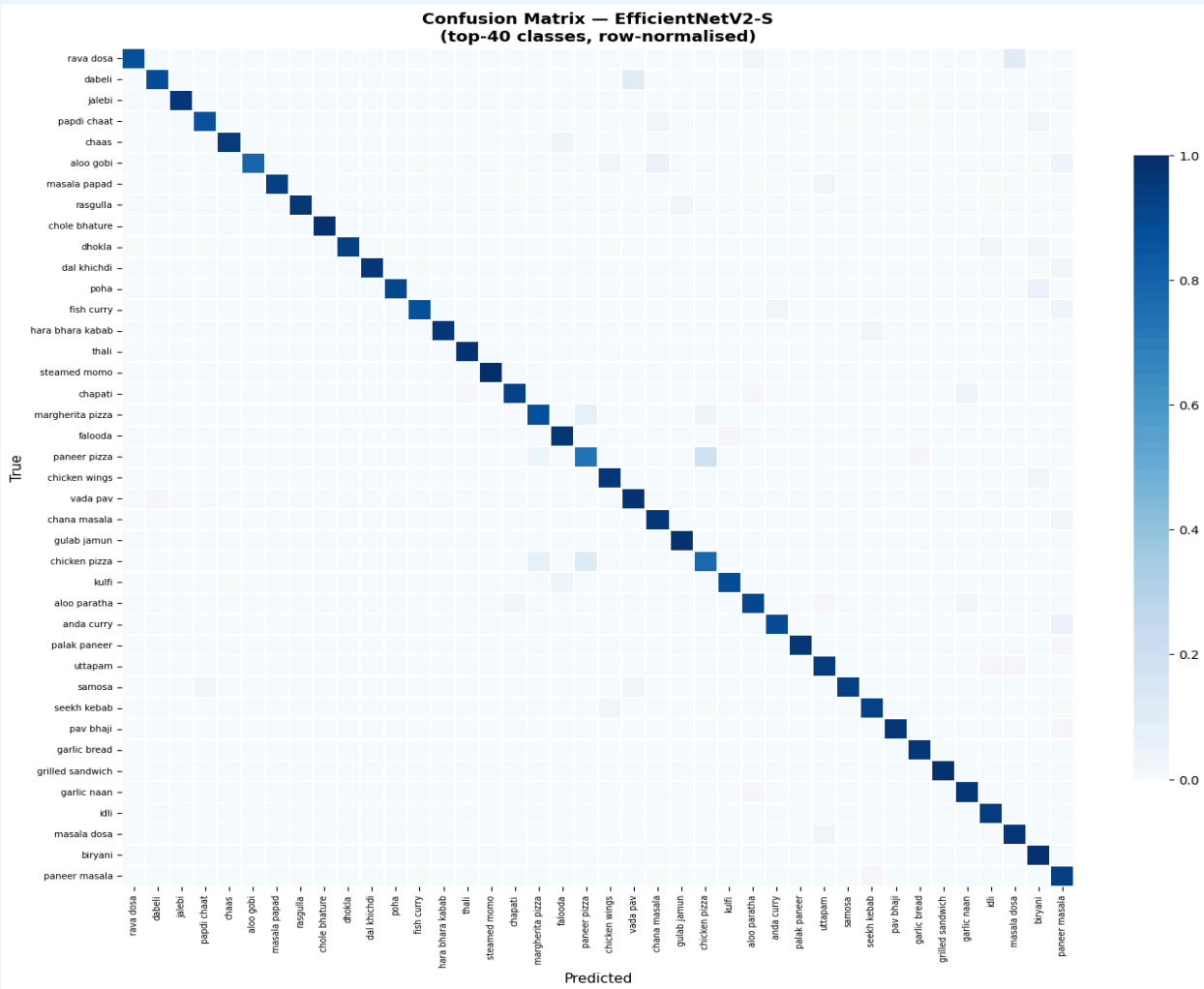


Figure 13: *Confusion matrix — EfficientNetV2-S (top 40 classes, row-normalised)* ▸ Insert from: **milestone5_evaluation_FIXED__5_.ipynb** *[EfficientNetV2-S confusion matrix cell (Milestone 5, Section 6.1.1)]*

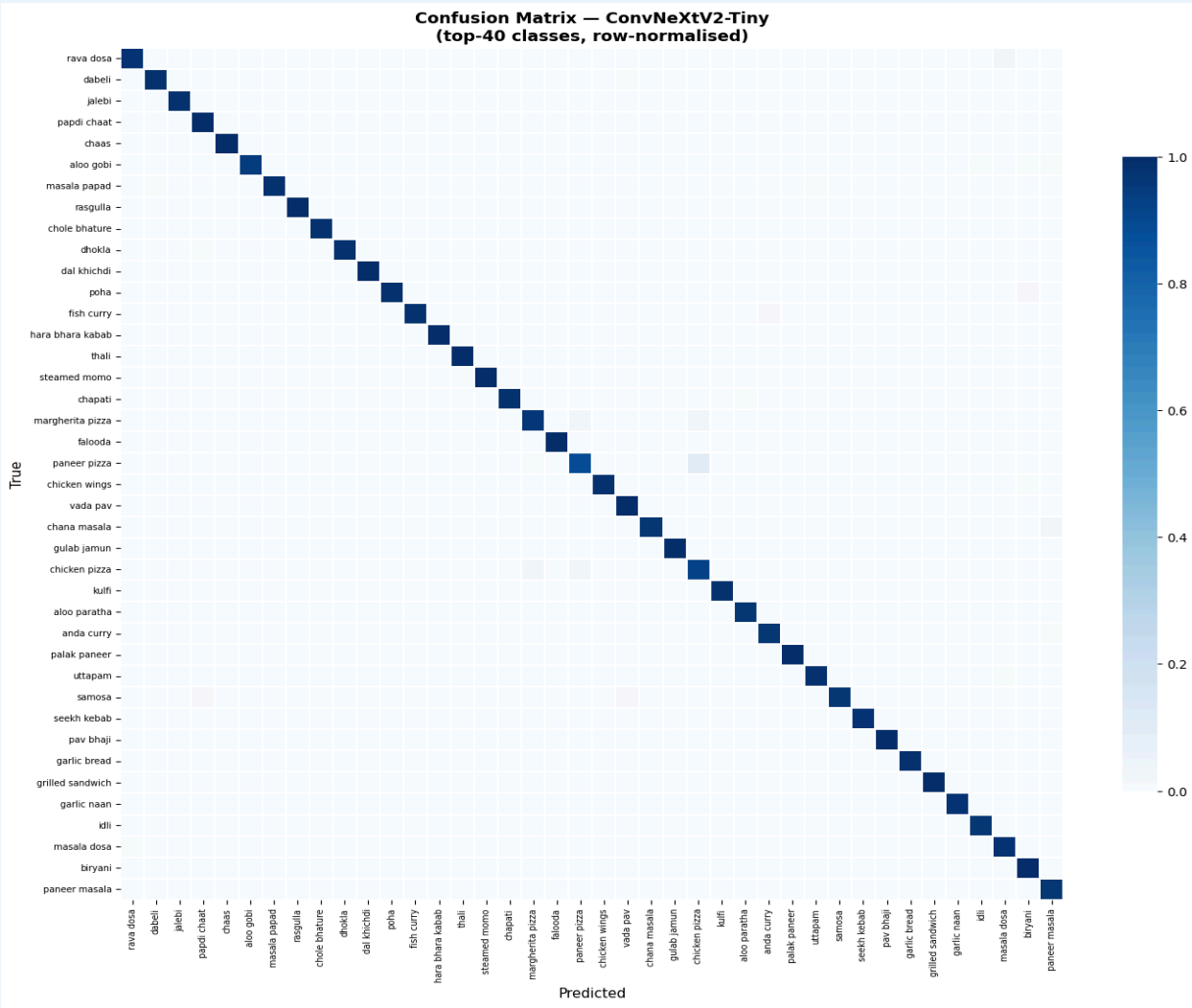


Figure 14: *Confusion matrix — ConvNeXtV2-Tiny (top 40 classes, row-normalised)*

12.3 ROC-AUC & Precision-Recall Curves

Micro-averaged ROC curves show both models well above the random baseline. EfficientNetV2-S achieves micro AUC = 0.999; ConvNeXtV2-Tiny and the ensemble achieve micro AUC ≈ 1.000. Macro-averaged AUC returned NaN due to numerical edge cases with 80 imbalanced classes and was therefore excluded from the summary table. Micro-averaged precision-recall AP scores: EfficientNetV2-S = 0.970, ConvNeXtV2-Tiny = 0.992, Ensemble = 0.992.

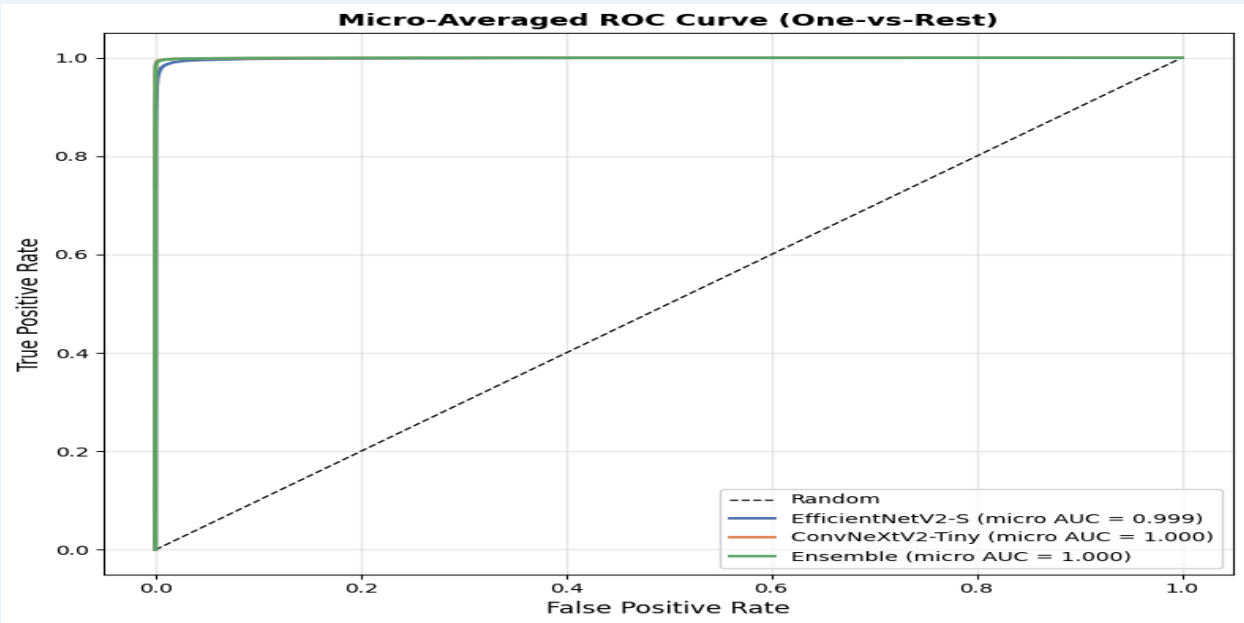


Figure 15: *Micro-averaged ROC curves for all three model configurations*

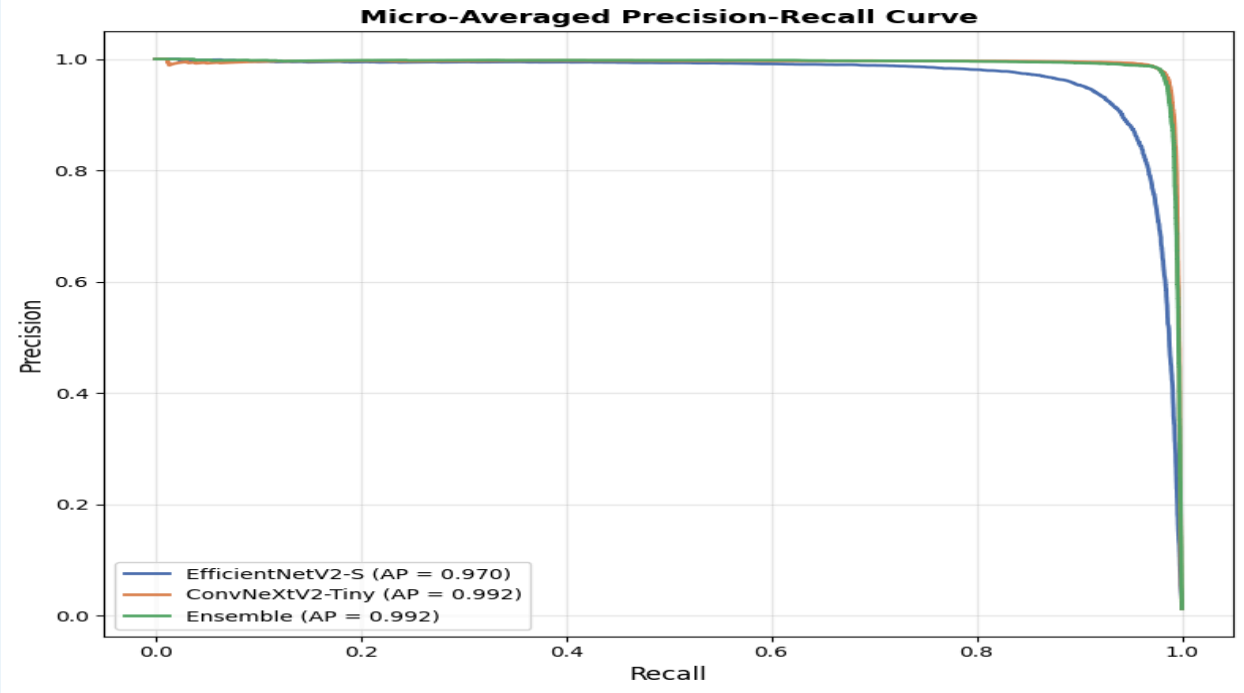


Figure 16: *Micro-averaged Precision-Recall curves for all three model configurations*

12.4 Per-Class Performance

EfficientNetV2-S — strongest classes: steamed momo (F1=0.994), sabudana vada (0.989), grilled sandwich (0.983). Weakest: amritsari kulcha (0.486), phirni (0.645), moong dal halwa (0.686). ConvNeXtV2-Tiny achieves uniformly high scores; fewer than a handful of classes fall below F1=0.90, all involving pizza variants or very low-sample classes.

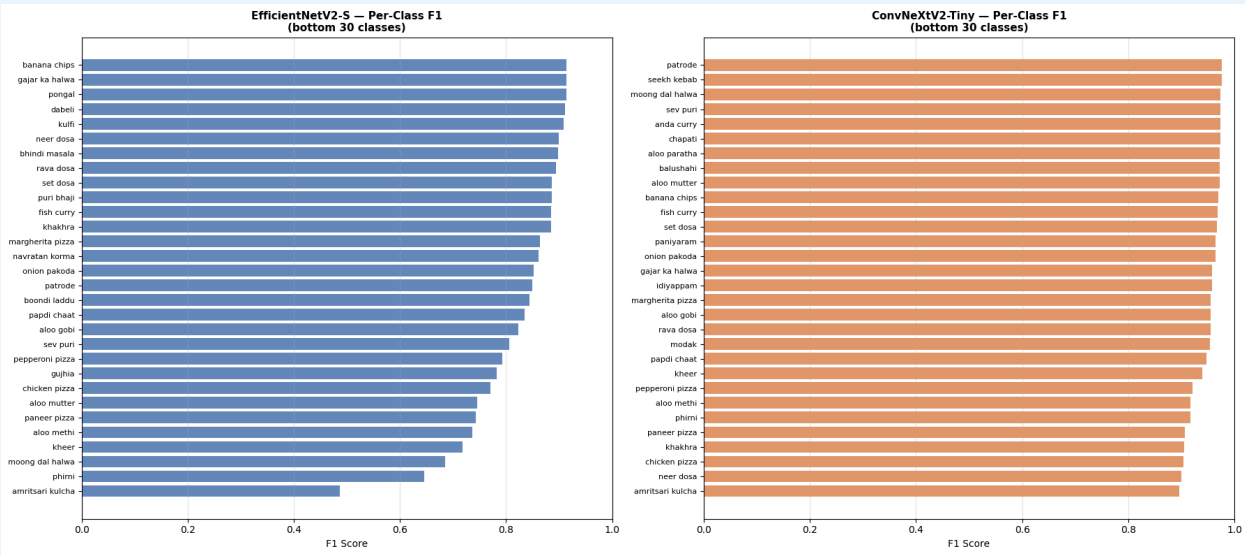


Figure 17: *Per-class F1 bar charts — bottom 30 classes for EfficientNetV2-S and ConvNeXtV2-Tiny*

12.5 Throughput & Latency

Model	Params (M)	Image Size	Inf Time (s)	Images/sec
EfficientNetV2-S	20.3	300 px	120.8 s / 13K imgs	109.2
ConvNeXtV2-Tiny	27.9	176 px	87.5 s / 13K imgs	150.7

13. Generalisation & Optimisation Techniques

Multiple techniques were combined across the classification training pipelines to improve generalisation and training stability:

13.1 Two-Phase Fine-Tuning (EfficientNetV2-S)

Phase 1 froze the backbone for 3 epochs, training only the classification head at high LR to avoid corrupting pretrained representations. Phase 2 unfroze the entire network with a 100× lower backbone LR, allowing gradual adaptation. This strategy prevents catastrophic forgetting on the 80-class Indian food task.

13.2 Cosine LR Scheduling with Warmup

Both models used 1-epoch linear warmup followed by cosine decay. Warmup prevents large gradient updates that destabilise pretrained weights in the first epoch; cosine decay ensures smooth convergence without oscillation.

13.3 Label Smoothing

Cross-entropy loss with label smoothing ε = 0.1 distributed a small probability (0.1 / 80) to all non-target classes. This prevents the model from becoming overconfident and improves calibration — especially important for the 80-class setting where inter-class similarity is high.

13.4 Weighted Random Sampling

The Khana dataset has extreme class imbalance (max 7,000 vs min <20 images per class). ConvNeXtV2's training used a weighted random sampler to ensure each class was seen proportionally, preventing bias toward high-frequency dishes like biryani.

13.5 Mixed Precision Training (AMP)

Both models used torch.amp.autocast for float16 forward passes with float32 gradient accumulation, reducing GPU memory by ~40–50% and accelerating training on Tesla T4 Tensor Cores.

13.6 Gradient Accumulation

A true batch size of 256 was infeasible on the T4. Gradient accumulation (4 × physical batch of 64) simulated effective batch 256 without out-of-memory risk. This was the single highest-impact hyperparameter intervention in the grid search (+3.44 Val F1 points over the batch-64 baseline).

13.7 MixUp and CutMix (EfficientNetV2-S)

Applied during Phase 2 full fine-tuning. MixUp linearly interpolates two training images and their labels; CutMix pastes a rectangular crop from one image into another. Both encourage soft predictions and improve calibration without changing the dataset size.

13.8 Stochastic Depth (EfficientNetV2-S)

Randomly drops residual branches during training, acting as an implicit ensemble over sub-networks and reducing overfitting in deep networks.

14. Challenges Faced & How They Were Resolved

Challenge	Root Cause
C1 — ConvNeXtV2 appeared to predict single class for all inputs (M4)	Wrong image size (224px) and wrong normalisation during evaluation
C2 — CUDA OOM on high-resolution phone photos with 4x SAM3 calls	3000–4000px phone photos exceed SAM3 native resolution
C3 — LLM chose arbitrary density values, causing weight errors	Density not explicitly provided in prompt; LLM guessed from training knowledge
C4 — LLM parse failures crash the whole pipeline for all items	Unstructured LLM response fails Pydantic schema validation

C5 — Double-counting food items (same food segmented by 2 prompts)	SAM3 run with 3 different prompts; all returning overlapping masks
C6 — Gradio streaming not working (only final state shown)	demo.queue() not called before demo.launch()
C7 — Session files accumulating on disk over time	Temporary directories not cleaned up between sessions
C8 — Annotated image uses wrong dimensions (mask vs. original image size mismatch)	annotated image generated from image_pil but masks derived from resized SAM image

15. Key Findings, Limitations

15.1 Key Findings

- ConvNeXtV2-Tiny is the strongest single classifier: Top-1 97.92%, Macro F1 0.9746, mAP 0.9859 — outperforming EfficientNetV2-S by ~5% accuracy on this dataset.
- The ensemble marginally improves over ConvNeXtV2-Tiny alone (+0.08% Top-1) because both models make complementary errors on only a small fraction of images.
- Effective batch size (via gradient accumulation) is the most impactful hyperparameter: +3.44 Val F1 points over batch-64 baseline.
- Conservative backbone LR is critical: reducing from 1e-4 to 5e-5 gave +3.30 points; increasing to 3e-4 caused catastrophic forgetting (-6.57 points).
- Preprocessing bugs caused the most misleading result in the project: incorrect image size and normalisation made ConvNeXtV2-Tiny appear near-random when it was actually the best model.
- The PCA + coin-calibration weight estimation approach is more practical than depth estimation or MLP regression for Indian food images.
- Top-5 accuracy of 99.53–99.76% demonstrates that the models' learned representations are semantically well-organised: the correct class is almost always within the top 5 predictions.

15.2 Limitations

- Pizza-variant confusion (chicken/paneer/margherita) is the dominant error pattern for both models — visually irreducible with 2D images alone.
- Minority classes with fewer than 30 test samples (e.g., aloo methi: 12, neer dosa: 21) show high metric variance and limited reliability.
- Dataset diversity may not cover all real-world photography conditions (overhead shots, partial occlusion, dim lighting, phone-camera lens distortion).
- PCA weight estimation is approximate: accuracy depends on clean segmentation masks, correct food shape assumptions, and absence of garnishes or partial occlusion.
- Macro ROC-AUC numerical issues with 80 imbalanced classes; micro-AUC used as substitute.

16. Future Improvements

16.1 Hyperparameter Tuning (Currently Incomplete)

📌 **Status:** Hyperparameter tuning was initiated (16 grid search experiments + 1 Optuna trial) but the tuned model was only retrained for 1 epoch due to compute constraints. The intended full retraining (3–5 epochs with BS_256_GA) was not completed within the project timeline.

Improvement	Expected Gain
Full retraining with BS_256_GA config (5+ epochs)	Expected +2–3% Top-1 over current 97.92% baseline; grid search showed +3.44 Val F1 pts
Reduce backbone LR from 1e-4 to 5e-5	Second most impactful factor (+3.30 Val F1 pts in grid search)
Test-Time Augmentation (5-crop TTA)	Expected +1–2% Top-1 accuracy at inference time with no retraining
Self-supervised pretraining on Indian food images	Closes domain gap from ImageNet; beneficial for rare classes with <30 images

16.2 Classification Improvements

- Expand dataset to 150+ food classes — add regional specialties, street foods, sweets not yet covered.
- Targeted data collection for currently weak classes: amritsari kulcha (F1=0.486), phirni (F1=0.645), moong dal halwa (F1=0.686).
- Pizza-variant disambiguation: chicken/paneer/margherita pizza share near-identical visual features. Ingredient-level annotations or multi-view input are needed.
- Synthetic data augmentation via diffusion models for rare classes with fewer than 50 training images.
- Ensemble of ConvNeXtV2 and EfficientNetV2 for production deployment (+0.08% Top-1 accuracy improvement).

16.3 Weight Estimation Improvements

- Replace geometric approximation with real-time monocular depth estimation (ZoeDepth or Depth Anything V2) for non-flat, volumetric foods like gulab jamun, rasgulla, and modak.
- Multi-view weight estimation: use front + top photograph to disambiguate height of tall glasses (lassi, chaas) which have similar 2D footprints to flat dishes.
- Collect a larger custom weight dataset (target: 500+ images, 80 classes) to train a proper regression model as backup to geometric estimation.
- Improve coin detection robustness — currently fails if coin is tilted, occluded, or at image edge. Add YOLO-based coin detector as fallback.

16.4 Application & Deployment Improvements

- Mobile app (React Native + TensorFlow Lite) for on-device classification, reducing latency from ~8s to <2s.
- Multi-meal tracking: accumulate detected items across a day and compare against ICMR daily RDA totals.
- User profile onboarding: age, gender, activity level for personalised ICMR RDA targets (currently hardcoded).
- Full LLM dietary suggestion integration with MedGemma safety layer — currently the app shows weight reasoning but not full dietary suggestions.
- Barcode scanning fallback for packaged foods not in the 80-class image classifier.
- Support for non-coin calibration: e.g. credit card (85.6mm × 53.98mm) or standard plate diameter as alternative reference objects.
- Model quantisation (INT8/FP16 via TensorRT) for faster inference on smaller GPUs.

16.5 Data & Nutrition Improvements

- Replace static per-100g values with cooking-method-aware values (e.g. biryani fried vs. steamed has different fat content).
- Add regional nutrition variation — Chettinad biryani vs. Kolkata biryani have meaningfully different macronutrient profiles.
- Integrate with ICMR-NIN database API for real-time nutritional value updates rather than a static JSON.
- Add micronutrient tracking: vitamin D, vitamin B12, zinc, magnesium — important for Indian vegetarian diets.

17. Conclusion

NutriVision successfully demonstrates a complete, production-deployed, end-to-end AI pipeline for Indian food nutrition estimation. The system evolved substantially across five milestones — from a YOLO-based detection-first approach to a SAM3 segmentation-first architecture with dedicated ConvNeXtV2/EfficientNetV2 classification backbones and PCA-based geometric weight estimation, and from a static nutrition table to a fully streaming Gradio web application with LLM reasoning.

The most significant technical achievements are: (1) ConvNeXtV2-Tiny achieving 97.92% Top-1 accuracy and 0.9746 Macro F1 across 80 fine-grained Indian food classes, with the ensemble reaching 98.00% — establishing the system as a strong foundation for practical food logging and calorie estimation; (2) a novel coin-calibrated PCA geometry pipeline that converts 2D pixel

measurements to real-world dimensions without depth sensors; (3) a density-enforced LLM weight estimation approach that decouples geometric reasoning from density constants, proving practically superior to regression approaches when labelled data is scarce; and (4) a fully streaming Gradio application with session isolation and step-by-step progress feedback, deployed at https://7860-01kp8gvr84ng3myeemyw7vzz46.cloudspaces.litng.ai.

The most valuable lessons from the project include the critical importance of per-model preprocessing in evaluation, the outsized impact of effective batch size and conservative backbone learning rates in fine-tuning, and the practical advantages of geometry-based weight estimation over learned regression. The modular codebase (config.py, models.py, pipeline.py, app.py) ensures every component is independently upgradeable, making NutriVision a robust platform for further development tailored to Indian cuisine.

The most impactful pending improvements are completing the hyperparameter tuning retraining (expected +2–3% further accuracy gain), adding monocular depth estimation for volumetric foods, and building a full personalised dietary recommendation pipeline with ICMR RDA comparison against the user's daily intake profile.